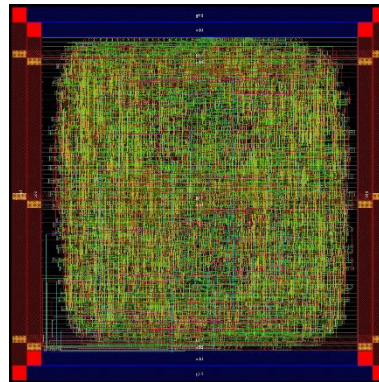
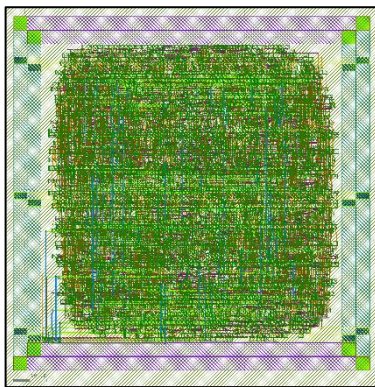


Computer-aided Digital Integrated Circuit Design (ECE748)

Home Assignment 4: Gate and Physical Synthesis



Course Instructor:

Dr. A. Abdelhadi

Students:

Ali Abbasi (abbasa46)
Derek Furtado (furtadod)

Fall 2023

Part 1: Synthesis with FreePDK45 Library

Problem 1.1 – SRAM Memory Compiler

Use the OpenRAM SRAM compiler to generate SRAM blocks for your RISC-V design. If the generated file format is “.lib”, convert it to “.db” format using Synopsys Library Compiler.

There are two memory modules in the RISC-V design provided in Assignment 1, one of which is Instruction Memory, and the other is Data Memory. We generate an SRAM model for both modules with the configuration shown in Code 1.

Code 1. SRAM configuration file (SRAM_32x64_1rw.py)

```
num_rw_ports    = 1
num_r_ports     = 0
num_w_ports     = 0

word_size       = 32
num_words       = 128
num_banks       = 1
words_per_row   = 4

tech_name       = "freepdk45"
process_corners = ["TT"]
supply_voltages = [1.1]
temperatures    = [25]

route_supplies  = True
check_lvsdrc    = True

output_path     = "SRAM_32x128_1rw"
output_name     = "SRAM_32x128_1rw"
instance_name   = "SRAM_32x128_1rw"
```

It is worth noting that the address width in the used RISC-V design is 32 bits. However, the generated SRAM using OpenRAM is much smaller and has only 6 bits of address. We ignored this difference since we knew we would not use any invalid addresses. Consequently, the extra bits in the address are dropped from the Most Significant Bit (MSB) side.

In order to use the OpenRAM, we clone the repository using the command below:

```
git clone https://github.com/VLSIDA/OpenRAM.git
```

After that, we enter the OpenRAM directory and set the following environment variables:

```
export OPENRAM_HOME="$HOME/OpenRAM/compiler"
export OPENRAM_TECH="$HOME/OpenRAM/technology"
export PYTHONPATH="$PYTHONPATH:$OPENRAM_HOME"
```

Now, we can invoke the following command with the configuration file in Code 1 passed to generate the SRAM:

```
python3 $OPENRAM_HOME/./sram_compiler.py SRAM_32x64_1rw.py
```

This command will automatically install Conda, create a virtual environment, install dependencies, and generate the SRAM model. The output files will be stored in a folder with the same name as the configuration name, “SRAM_32x64_1rw”. The tool will generate all required files, such as the “.lib” file for the library, a Verilog module with a behavioural implementation of the SRAM to be used for simulation, and the GDS file. Figure 1. Shows the stick diagram for the generated SRAM.

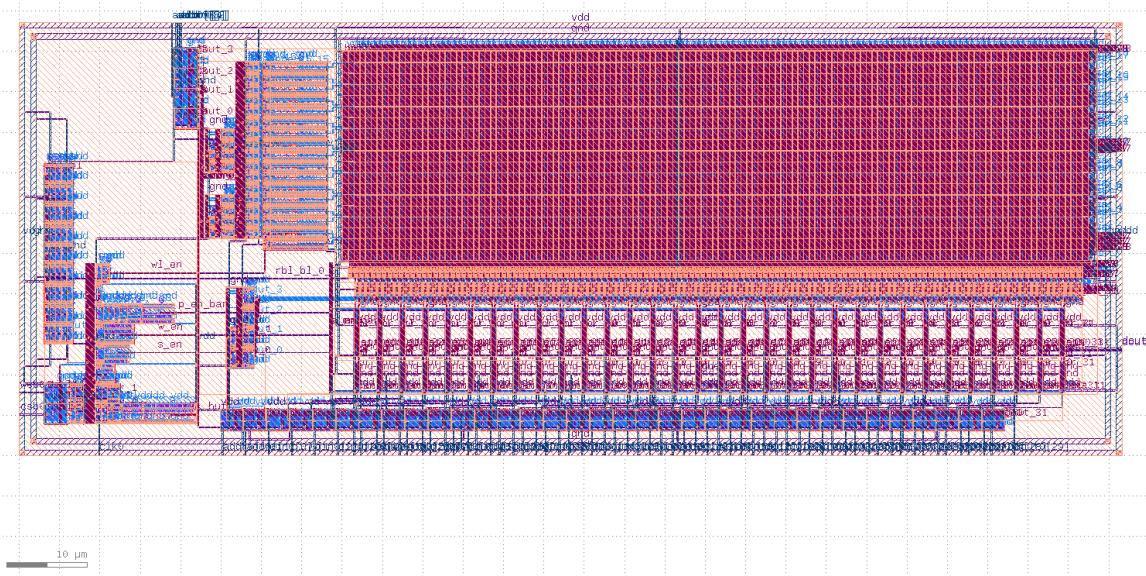


Figure 1. Stick diagram of the SRAM generated by OpenRAM based on the SRAM_32x64_1rw configuration.

As expected, the generated library file has a “.lib” format, which cannot be used for the next stages of the project. In order to convert it to “.db” format, Synopsys Library Compiler is used with the TCL script shown in Code 2, which converts all generated SRAM models to “.db” format.

Code 2. Script to translate generated SRAM libraries to “.db” (lc_shell_translate_sram_all.tcl)

```
# source Synopsys Library Compiler
# source /CMC/scripts/synopsys.lc.2022.12.csh
#
# run lc_shell on command line with this tcl script passed:
# lc_shell -f translate_sram_models.tcl
# if dc_shell is already running:
# source translate_sram_models.tcl

read_lib SRAM_32x64_1rw_FF_1p0V_25C.lib
write_lib SRAM_32x64_1rw_FF_1p0V_25C_lib -format db -output
SRAM_32x64_1rw_FF_1p0V_25C.db
read_lib SRAM_32x64_1rw_SS_1p0V_25C.lib
write_lib SRAM_32x64_1rw_SS_1p0V_25C_lib -format db -output
SRAM_32x64_1rw_SS_1p0V_25C.db
read_lib SRAM_32x64_1rw_TT_1p0V_25C.lib
write_lib SRAM_32x64_1rw_TT_1p0V_25C_lib -format db -output
SRAM_32x64_1rw_TT_1p0V_25C.db
read_lib SRAM_32x64_1rw_TT_1p1V_25C.lib
write_lib SRAM_32x64_1rw_TT_1p1V_25C_lib -format db -output
SRAM_32x64_1rw_TT_1p1V_25C.db
```

After the commands above are completed, the “.db” files are generated in the same directory.

Problem 1.2.1 – Gate Synthesis using Synopsys Design Compiler

Perform a complete gate synthesis on your RISC-V design.

Before running the synthesis process using the Design Compiler, we had to modify the RISC -V HDL codes based on the requirements of the assignment. The first modification was to add VCC and GND ports as

INOUT to all modules in the design. Secondly, we have replaced the instruction and data memory with the SRAM Verilog model generated by OpenRAM. The generated SRAM was sensitive to the negative edge of the clock, while the whole RISC-V design was positive edge-sensitive. Therefore, we modified the RISC-V to work with the negative edge of the clock. Finally, we replaced the adder/subtractor module used in the ALU with the IP provided in Synopsys DesignWare.

After applying the mentioned modifications, we followed the steps in the assignment description and developed the TCL script shown in Code 3. This script must run inside the dc_shell after sourcing the Synopsys Design Compiler. It will start with specifying libraries, reading design files, setting up the constraints, and performing the compilation. Additionally, it generates the SDF, SDC, and netlist files at the end, which are needed for the post-synthesis simulation. All synthesis outputs are accessible at the following path: “./part_1/04_syn_freepdk45/synthesis_outputs/”.

Code 3. Script to compile the design and generate timing files (dc_shell_syn_freepdk45.tcl)

```
# source Synopsys Design Compiler
# source /CMC/scripts/synopsys.syn.2022.12.csh
#
# run dc_shell on command line with this tcl script passed
# dc_shell -f dc_shell_syn_freepdk45.tcl
# if dc_shell is already running
# source dc_shell_syn_freepdk45.tcl

# specify libraries
set target_library "~/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_1rw_TT_1p1V_25C.db
~/mydata/04_syn_freepdk45/pdk_files/gscl45nm.db"
set symbol_library "generic.sdb"
set synthetic_library "dw_foundation.sldb standard.sldb"
set link_library "~/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_1rw_TT_1p1V_25C.db
~/mydata/04_syn_freepdk45/pdk_files/gscl45nm.db dw_foundation.sldb standard.sldb"

# read design
# analysis and elaboration are done automatically
read_file {./rtl/sources_1} -autoread -format verilog -top Single_Cycle_Top

# set design environment
set_operating_conditions "typical"
set_drive 0.01 [all_inputs]
set_load 0.01 [all_outputs]

# set clock constraints
create_clock -period 2.6 clk
set_clock_latency 0.01 clk
set_input_delay 0.02 -clock clk [all_inputs]
set_output_delay 0.01 -clock clk [all_outputs]

# set design constraints
set_max_transition 2000 [get_designs Single_Cycle_Top]
set_max_fanout 100 [all_inputs]
set_fanout_load 0.001 [all_outputs]
set_high_fanout_net_threshold 100
set_max_capacitance 50000 [all_outputs]
set_max_area 50000

# resolve multiple refereneces
uniquify

# compile
compile
```

```
# check design for errors/warnings
# uplevel #0 check_design

# check all i/o properties
# report_port

# report design metrics
# report_area
# report_timing
# report_power
# report_constraint

# generate sdc file
write_sdc ./Single_Cycle_Top_syn.sdc -version 1.8

# generate sdf file
write_sdf -version 2.1 -context verilog -load_delay cell ./Single_Cycle_Top_syn.sdf

# generate verilog netlist
write -hierarchy -format verilog -output ./Single_Cycle_Top_syn.v
```

Throughout the compilation process, no error was received for the used RISC-V code. However, some warnings stated that some bits of the instruction memory and data memory address are unconnected. This warning is because of the address width difference between the generated SRAM and the RISC-V design, as explained in the previous part. We consider this error as non-critical as it will not affect the code's functionality as long as we do not access the invalid address range provided by a 32-bit address line, which is not the case since we already know the test case of the design.

Problem 1.2.2 – Critical Path

In order to check the first five critical paths, we can invoke the following command in the dc_shell after the compilation is completed:

```
report_timing -delay_type max -max_path 5
```

By running the command above, five tables are printed in the terminal, each stating the exact nodes each critical path traverses. These tables are shown in Figures 2 to 6. The most critical path travels from Instruction Memory, going through the ALU, and entering the Data Memory through its write data inputs. We will discuss the critical path later on in-depth when proposing timing improvement techniques in Problem 1.2.4.

Problem 1.2.3 – False Paths and Multi-cycle Paths

A simple unexercised false path in our design can be the path from ALU to the Data Memory module. Although there are some instructions in the ISA that end up in the write-back stage, it does not happen in our pre-written assembly code. However, the design is supposed to be a general-purpose processor capable of running every assembly code. If we want to be more general, another example of an unexercised path can be the path that is traversed once an interrupt is happened.

In case of multi-cycle paths, our current design does not have a multi-cycle path as it is a single-cycle processor. However, if we could perform pipelining on our design for timing improvement, an example of a multi-cycle path could be the path that data traverses during forwarding to address the hazards.

Startpoint: Instruction_Memory/clk0 (internal path startpoint clocked by clk)		
Endpoint: Data_Memory (rising edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Point	Incr	Path

clock clk (fall edge)	1.30	1.30
clock network delay (ideal)	0.01	1.31
input external delay	0.00	1.31 f
Instruction_Memory/clk0 (SRAM_32x64_1rw)	0.00 #	1.31 f
Instruction_Memory/dout0[21] (SRAM_32x64_1rw)	0.30	1.61 r
Register_File/A2[1] (Register_File)	0.00	1.61 r
Register_File/U60/Y (INVX2)	0.03	1.64 f
Register_File/U59/Y (INVX1)	0.00	1.65 r
Register_File/U1582/Y (NAND3X1)	0.01	1.66 f
Register_File/U462/Y (BUF2)	0.06	1.72 f
Register_File/U404/Y (INVX1)	0.02	1.74 r
Register_File/U367/Y (INVX1)	0.02	1.75 f
Register_File/U21/Y (INVX1)	0.12	1.88 r
Register_File/U2773/Y (NAND2X1)	0.05	1.92 f
Register_File/U2775/Y (NAND3X1)	0.04	1.96 r
Register_File/U2783/Y (OAI21X1)	0.02	1.98 f
Register_File/U2799/Y (OAI21X1)	0.04	2.02 r
Register_File/U329/Y (INVX1)	0.02	2.05 f
Register_File/RD2[19] (Register_File)	0.00	2.05 f
Mux_Register_to_ALU/a[19] (Mux_1)	0.00	2.05 f
Mux_Register_to_ALU/U56/Y (MUX2X1)	0.05	2.09 r
Mux_Register_to_ALU/U23/Y (INVX2)	0.03	2.12 f
Mux_Register_to_ALU/c[19] (Mux_1)	0.00	2.12 f
ALU/B[19] (ALU)	0.00	2.12 f
ALU/DW01_addsub_inst/B[19] (ALU_DW01_addsub_1)	0.00	2.12 f
ALU/DW01_addsub_inst/U387/Y (XOR2X1)	0.06	2.18 r
ALU/DW01_addsub_inst/U420/Y (OR2X1)	0.04	2.22 r
ALU/DW01_addsub_inst/U498/Y (INVX1)	0.02	2.25 f
ALU/DW01_addsub_inst/U421/Y (OR2X2)	0.04	2.29 f
ALU/DW01_addsub_inst/U470/Y (INVX1)	0.01	2.29 r
ALU/DW01_addsub_inst/U129/Y (NAND2X1)	0.02	2.31 f
ALU/DW01_addsub_inst/U128/Y (OAI21X1)	0.06	2.37 r
ALU/DW01_addsub_inst/U521/Y (INVX4)	0.03	2.39 f
ALU/DW01_addsub_inst/U601/Y (INVX1)	0.02	2.41 r
ALU/DW01_addsub_inst/U14/Y (AOI21X1)	0.01	2.42 f
ALU/DW01_addsub_inst/U525/Y (NAND2X1)	0.03	2.45 r
ALU/DW01_addsub_inst/U3/Y (XNOR2X1)	0.05	2.50 f
ALU/DW01_addsub_inst/SUM[31] (ALU_DW01_addsub_1)	0.00	2.50 f
ALU/U71/Y (NAND3X1)	0.04	2.54 r
ALU/U52/Y (NAND2X1)	0.02	2.56 f
ALU/U51/Y (BUF2)	0.04	2.60 f
ALU/Result[0] (ALU)	0.00	2.60 f
Data_Memory/addr0[0] (SRAM_32x64_1rw)	0.00	2.60 f
data arrival time		2.60
clock clk (rise edge)	2.60	2.60
clock network delay (ideal)	0.01	2.61
Data_Memory/clk0 (SRAM_32x64_1rw)	0.00	2.61 r
library setup time	-0.01	2.60
data required time		2.60

data required time		2.60
data arrival time		-2.60

slack (MET)		0.00

Figure 2. 1st critical path in the design synthesized using FreePDK45 with a clock period of 2.6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[2]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 r
rst (in)	0.00	0.03 r
Register_File/rst (Register_File)	0.00	0.03 r
Register_File/U29/Y (AND2X2)	0.04	0.07 r
Register_File/U80/Y (INVX1)	0.02	0.09 f
Register_File/U2603/Y (NOR2X1)	0.03	0.12 r
Register_File/U2604/Y (NAND2X1)	0.02	0.14 f
Register_File/U2609/Y (NOR2X1)	0.03	0.17 r
Register_File/U2610/Y (AOI21X1)	0.02	0.19 f
Register_File/U2611/Y (NAND2X1)	0.02	0.20 r
Register_File/RD2[16] (Register_File)	0.00	0.20 r
Mux_Register_to_ALU/a[16] (Mux_1)	0.00	0.20 r
Mux_Register_to_ALU/U54/Y (MUX2X1)	0.03	0.23 f
Mux_Register_to_ALU/U13/Y (INVX2)	0.04	0.26 r
Mux_Register_to_ALU/c[16] (Mux_1)	0.00	0.26 r
ALU/B[16] (ALU)	0.00	0.26 r
ALU/DW01_addsub_inst/B[16] (ALU_DW01_addsub_1)	0.00	0.26 r
ALU/DW01_addsub_inst/U390/Y (XOR2X1)	0.03	0.30 f
ALU/DW01_addsub_inst/U207/Y (NOR2X1)	0.03	0.33 r
ALU/DW01_addsub_inst/U454/Y (INVX1)	0.02	0.35 f
ALU/DW01_addsub_inst/U455/Y (INVX1)	0.01	0.36 r
ALU/DW01_addsub_inst/U204/Y (AOI21X1)	0.02	0.38 f
ALU/DW01_addsub_inst/U180/Y (AOI21X1)	0.04	0.42 r
ALU/DW01_addsub_inst/U551/Y (BUF2)	0.04	0.46 r
ALU/DW01_addsub_inst/U608/Y (INVX1)	0.02	0.48 f
ALU/DW01_addsub_inst/U152/Y (AOI21X1)	0.05	0.52 r
ALU/DW01_addsub_inst/U563/Y (INVX1)	0.02	0.55 f
ALU/DW01_addsub_inst/U148/Y (AOI21X1)	0.03	0.57 r
ALU/DW01_addsub_inst/U137/Y (XOR2X1)	0.04	0.61 r
ALU/DW01_addsub_inst/SUM[21] (ALU_DW01_addsub_1)	0.00	0.61 r
ALU/U153/Y (NAND2X1)	0.02	0.63 f
ALU/U154/Y (NAND3X1)	0.05	0.68 r
ALU/U55/Y (OR2X2)	0.05	0.73 r
ALU/U202/Y (NOR3X1)	0.03	0.76 f
ALU/U203/Y (NAND3X1)	0.03	0.79 r
ALU/U204/Y (NOR2X1)	0.02	0.82 f
ALU/Zero (ALU)	0.00	0.82 f
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/U12/Y (XNOR2X1)	0.03	0.84 f
Control_Unit_Top/Main_Decoder/U11/Y (NOR2X1)	0.03	0.87 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	0.87 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	0.87 r
Mux_PCNext/s (Mux_2)	0.00	0.87 r
Mux_PCNext/U15/Y (INVX1)	0.03	0.90 f
Mux_PCNext/U14/Y (INVX4)	0.04	0.94 r
Mux_PCNext/U53/Y (MUX2X1)	0.04	0.98 f
Mux_PCNext/c[2] (Mux_2)	0.00	0.98 f
PC/PC_Next[2] (PC_Module)	0.00	0.98 f
PC/U35/Y (AND2X1)	0.03	1.01 f
PC/PC_reg[2]/D (DFFNEGX1)	0.00	1.01 f
data arrival time		1.01
clock clk (fall edge)	1.30	1.30
clock network delay (ideal)	0.01	1.31
PC/PC_reg[2]/CLK (DFFNEGX1)	0.00	1.31 f
library setup time	-0.30	1.01
data required time		1.01
data required time		1.01
data arrival time		-1.01
slack (MET)		0.00

Figure 3. 2nd critical path in the design synthesized using FreePDK45 with a clock period of 2.6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[4]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 r
rst (in)	0.00	0.03 r
Register_File/rst (Register_File)	0.00	0.03 r
Register_File/U29/Y (AND2X2)	0.04	0.07 r
Register_File/U80/Y (INVX1)	0.02	0.09 f
Register_File/U2603/Y (NOR2X1)	0.03	0.12 r
Register_File/U2604/Y (NAND2X1)	0.02	0.14 f
Register_File/U2609/Y (NOR2X1)	0.03	0.17 r
Register_File/U2610/Y (AOI21X1)	0.02	0.19 f
Register_File/U2611/Y (NAND2X1)	0.02	0.20 r
Register_File/RD2[16] (Register_File)	0.00	0.20 r
Mux_Register_to_ALU/a[16] (Mux_1)	0.00	0.20 r
Mux_Register_to_ALU/U54/Y (MUX2X1)	0.03	0.23 f
Mux_Register_to_ALU/U13/Y (INVX2)	0.04	0.26 r
Mux_Register_to_ALU/c[16] (Mux_1)	0.00	0.26 r
ALU/B[16] (ALU)	0.00	0.26 r
ALU/DW01_addsub_inst/B[16] (ALU_DW01_addsub_1)	0.00	0.26 r
ALU/DW01_addsub_inst/U390/Y (XOR2X1)	0.03	0.30 f
ALU/DW01_addsub_inst/U207/Y (NOR2X1)	0.03	0.33 r
ALU/DW01_addsub_inst/U454/Y (INVX1)	0.02	0.35 f
ALU/DW01_addsub_inst/U455/Y (INVX1)	0.01	0.36 r
ALU/DW01_addsub_inst/U204/Y (AOI21X1)	0.02	0.38 f
ALU/DW01_addsub_inst/U180/Y (AOI21X1)	0.04	0.42 r
ALU/DW01_addsub_inst/U551/Y (BUF2X1)	0.04	0.46 r
ALU/DW01_addsub_inst/U608/Y (INVX1)	0.02	0.48 f
ALU/DW01_addsub_inst/U152/Y (AOI21X1)	0.05	0.52 r
ALU/DW01_addsub_inst/U563/Y (INVX1)	0.02	0.55 f
ALU/DW01_addsub_inst/U148/Y (AOI21X1)	0.03	0.57 r
ALU/DW01_addsub_inst/U137/Y (XOR2X1)	0.04	0.61 r
ALU/DW01_addsub_inst/SUM[21] (ALU_DW01_addsub_1)	0.00	0.61 r
ALU/U153/Y (NAND2X1)	0.02	0.63 f
ALU/U154/Y (NAND3X1)	0.05	0.68 r
ALU/U55/Y (OR2X2)	0.05	0.73 r
ALU/U202/Y (NOR3X1)	0.03	0.76 f
ALU/U203/Y (NAND3X1)	0.03	0.79 r
ALU/U204/Y (NOR2X1)	0.02	0.82 f
ALU/Zero (ALU)	0.00	0.82 f
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/U12/Y (XNOR2X1)	0.03	0.84 f
Control_Unit_Top/Main_Decoder/U11/Y (NOR2X1)	0.03	0.87 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	0.87 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	0.87 r
Mux_PCNext/s (Mux_2)	0.00	0.87 r
Mux_PCNext/U15/Y (INVX1)	0.03	0.90 f
Mux_PCNext/U14/Y (INVX4)	0.04	0.94 r
Mux_PCNext/U18/Y (MUX2X1)	0.04	0.98 f
Mux_PCNext/c[4] (Mux_2)	0.00	0.98 f
PC/PC_Next[4] (PC_Module)	0.00	0.98 f
PC/U6/Y (AND2X1)	0.03	1.01 f
PC/PC_reg[4]/D (DFFNEGX1)	0.00	1.01 f
data arrival time		1.01
clock clk (fall edge)	1.30	1.30
clock network delay (ideal)	0.01	1.31
PC/PC_reg[4]/CLK (DFFNEGX1)	0.00	1.31 f
library setup time	-0.30	1.01
data required time		1.01
data required time		1.01
data arrival time		-1.01
slack (MET)		0.00

Figure 4. 3rd critical path in the design synthesized using FreePDK45 with a clock period of 2.6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[3]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 r
rst (in)	0.00	0.03 r
Register_File/rst (Register_File)	0.00	0.03 r
Register_File/U29/Y (AND2X2)	0.04	0.07 r
Register_File/U80/Y (INVX1)	0.02	0.09 f
Register_File/U2603/Y (NOR2X1)	0.03	0.12 r
Register_File/U2604/Y (NAND2X1)	0.02	0.14 f
Register_File/U2609/Y (NOR2X1)	0.03	0.17 r
Register_File/U2610/Y (AOI21X1)	0.02	0.19 f
Register_File/U2611/Y (NAND2X1)	0.02	0.20 r
Register_File/RD2[16] (Register_File)	0.00	0.20 r
Mux_Register_to_ALU/a[16] (Mux_1)	0.00	0.20 r
Mux_Register_to_ALU/U54/Y (MUX2X1)	0.03	0.23 f
Mux_Register_to_ALU/U13/Y (INVX2)	0.04	0.26 r
Mux_Register_to_ALU/c[16] (Mux_1)	0.00	0.26 r
ALU/B[16] (ALU)	0.00	0.26 r
ALU/DW01_addsub_inst/B[16] (ALU_DW01_addsub_1)	0.00	0.26 r
ALU/DW01_addsub_inst/U390/Y (XOR2X1)	0.03	0.30 f
ALU/DW01_addsub_inst/U207/Y (NOR2X1)	0.03	0.33 r
ALU/DW01_addsub_inst/U454/Y (INVX1)	0.02	0.35 f
ALU/DW01_addsub_inst/U455/Y (INVX1)	0.01	0.36 r
ALU/DW01_addsub_inst/U204/Y (AOI21X1)	0.02	0.38 f
ALU/DW01_addsub_inst/U180/Y (AOI21X1)	0.04	0.42 r
ALU/DW01_addsub_inst/U551/Y (BUF2)	0.04	0.46 r
ALU/DW01_addsub_inst/U608/Y (INVX1)	0.02	0.48 f
ALU/DW01_addsub_inst/U152/Y (AOI21X1)	0.05	0.52 r
ALU/DW01_addsub_inst/U563/Y (INVX1)	0.02	0.55 f
ALU/DW01_addsub_inst/U148/Y (AOI21X1)	0.03	0.57 r
ALU/DW01_addsub_inst/U137/Y (XOR2X1)	0.04	0.61 r
ALU/DW01_addsub_inst/SUM[21] (ALU_DW01_addsub_1)	0.00	0.61 r
ALU/U153/Y (NAND2X1)	0.02	0.63 f
ALU/U154/Y (NAND3X1)	0.05	0.68 r
ALU/U55/Y (OR2X2)	0.05	0.73 r
ALU/U202/Y (NOR3X1)	0.03	0.76 f
ALU/U203/Y (NAND3X1)	0.03	0.79 r
ALU/U204/Y (NOR2X1)	0.02	0.82 f
ALU/Zero (ALU)	0.00	0.82 f
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/U12/Y (XNOR2X1)	0.03	0.84 f
Control_Unit_Top/Main_Decoder/U11/Y (NOR2X1)	0.03	0.87 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	0.87 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	0.87 r
Mux_PCNext/s (Mux_2)	0.00	0.87 r
Mux_PCNext/U15/Y (INVX1)	0.03	0.90 f
Mux_PCNext/U14/Y (INVX4)	0.04	0.94 r
Mux_PCNext/U17/Y (MUX2X1)	0.04	0.98 f
Mux_PCNext/c[3] (Mux_2)	0.00	0.98 f
PC/PC_Next[3] (PC_Module)	0.00	0.98 f
PC/U5/Y (AND2X1)	0.03	1.01 f
PC/PC_reg[3]/D (DFFNEGX1)	0.00	1.01 f
data arrival time		1.01
clock clk (fall edge)	1.30	1.30
clock network delay (ideal)	0.01	1.31
PC/PC_reg[3]/CLK (DFFNEGX1)	0.00	1.31 f
library setup time	-0.30	1.01
data required time		1.01

data required time		1.01
data arrival time		-1.01

slack (MET)		0.00

Figure 5. 4th critical path in the design synthesized using FreePDK45 with a clock period of 2.6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[1]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 r
rst (in)	0.00	0.03 r
Register_File/rst (Register_File)	0.00	0.03 r
Register_File/U29/Y (AND2X2)	0.04	0.07 r
Register_File/U80/Y (INVX1)	0.02	0.09 f
Register_File/U2603/Y (NOR2X1)	0.03	0.12 r
Register_File/U2604/Y (NAND2X1)	0.02	0.14 f
Register_File/U2609/Y (NOR2X1)	0.03	0.17 r
Register_File/U2610/Y (AOI21X1)	0.02	0.19 f
Register_File/U2611/Y (NAND2X1)	0.02	0.20 r
Register_File/RD2[16] (Register_File)	0.00	0.20 r
Mux_Register_to_ALU/a[16] (Mux_1)	0.00	0.20 r
Mux_Register_to_ALU/U54/Y (MUX2X1)	0.03	0.23 f
Mux_Register_to_ALU/U13/Y (INVX2)	0.04	0.26 r
Mux_Register_to_ALU/c[16] (Mux_1)	0.00	0.26 r
ALU/B[16] (ALU)	0.00	0.26 r
ALU/DW01_addsub_inst/B[16] (ALU_DW01_addsub_1)	0.00	0.26 r
ALU/DW01_addsub_inst/U390/Y (XOR2X1)	0.03	0.30 f
ALU/DW01_addsub_inst/U207/Y (NOR2X1)	0.03	0.33 r
ALU/DW01_addsub_inst/U454/Y (INVX1)	0.02	0.35 f
ALU/DW01_addsub_inst/U455/Y (INVX1)	0.01	0.36 r
ALU/DW01_addsub_inst/U204/Y (AOI21X1)	0.02	0.38 f
ALU/DW01_addsub_inst/U180/Y (AOI21X1)	0.04	0.42 r
ALU/DW01_addsub_inst/U551/Y (BUF2)	0.04	0.46 r
ALU/DW01_addsub_inst/U608/Y (INVX1)	0.02	0.48 f
ALU/DW01_addsub_inst/U152/Y (AOI21X1)	0.05	0.52 r
ALU/DW01_addsub_inst/U563/Y (INVX1)	0.02	0.55 f
ALU/DW01_addsub_inst/U148/Y (AOI21X1)	0.03	0.57 r
ALU/DW01_addsub_inst/U137/Y (XOR2X1)	0.04	0.61 r
ALU/DW01_addsub_inst/SUM[21] (ALU_DW01_addsub_1)	0.00	0.61 r
ALU/U153/Y (NAND2X1)	0.02	0.63 f
ALU/U154/Y (NAND3X1)	0.05	0.68 r
ALU/U55/Y (OR2X2)	0.05	0.73 r
ALU/U202/Y (NOR3X1)	0.03	0.76 f
ALU/U203/Y (NAND3X1)	0.03	0.79 r
ALU/U204/Y (NOR2X1)	0.02	0.82 f
ALU/Zero (ALU)	0.00	0.82 f
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	0.82 f
Control_Unit_Top/Main_Decoder/U12/Y (XNOR2X1)	0.03	0.84 f
Control_Unit_Top/Main_Decoder/U11/Y (NOR2X1)	0.03	0.87 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	0.87 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	0.87 r
Mux_PCNext/s (Mux_2)	0.00	0.87 r
Mux_PCNext/U15/Y (INVX1)	0.03	0.90 f
Mux_PCNext/U14/Y (INVX4)	0.04	0.94 r
Mux_PCNext/U19/Y (MUX2X1)	0.04	0.98 f
Mux_PCNext/c[1] (Mux_2)	0.00	0.98 f
PC/PC_Next[1] (PC_Module)	0.00	0.98 f
PC/U9/Y (AND2X1)	0.03	1.01 f
PC/PC_reg[1]/D (DFFNEGX1)	0.00	1.01 f
data arrival time		1.01
clock clk (fall edge)	1.30	1.30
clock network delay (ideal)	0.01	1.31
PC/PC_reg[1]/CLK (DFFNEGX1)	0.00	1.31 f
library setup time	-0.30	1.01
data required time		1.01

data required time		1.01
data arrival time		-1.01

slack (MET)		0.00

Figure 6. 5th critical path in the design synthesized using FreePDK45 with a clock period of 2.6 ns

Problem 1.2.4 – Timing Improvement

In order to improve the timing, we can study paths with the lowest slack in the design. We expect that the path with the lowest slack is in the critical path of the design and can be the starting point for timing improvements. To do so, we use the TCL process shown in Code 4 that prints a specific number of paths sorted incrementally by their slack. The number of paths to report is passed as an argument to the process.

Code 4. TCL process to print a specific number of paths ordered incrementally by their slack (dc_shell_report_paths_slack.tcl)

```
proc custom_report_critical_paths {max_paths} {  
    echo [format "%-20s %-20s %7s" "From" "To" "Slack"]  
    echo "-----"  
    foreach_in_collection path [get_timing_paths -delay_type max -nworst $max_paths -  
max_paths $max_paths -include_hierarchical_pins] {  
        set slack [get_attribute $path slack]  
        set startpoint [get_attribute $path startpoint]  
        set endpoint [get_attribute $path endpoint]  
        echo [format "%-20s %-20s %s" [get_attribute $startpoint full_name] \  
            [get_attribute $endpoint full_name] $slack]  
    }  
}
```

The output of this process is shown in Figure 7.

```
dc_shell> custom_report_critical_paths 5  
From          To          Slack  
-----  
Instruction_Memory/clk0 Data_Memory/addr0[0] 1.19209e-05  
Instruction_Memory/clk0 Data_Memory/addr0[0] 9.40561e-05  
rst           PC/PC_reg[2]/D      0.000366569  
rst           PC/PC_reg[3]/D      0.000366569  
rst           PC/PC_reg[1]/D      0.000366569
```

Figure 7. The first five paths with the lowest slack time in the synthesis using FreePDK45

However, this process only shows the starting point and end point of the path. Therefore, it is better to use the reports used for the critical path in the previous part. If we look at the most critical path shown in Figure 2, we can see that the path from instruction memory output, the instruction itself, to the data memory has the highest delay in the design synthesized using FreePDK45nm. Along this path, the signal traverses through the multiplexer used to choose inputs of the Arithmetic and Logic Unit (ALU), and then, it goes through the most complex sub-module of the ALU. The ALU can only perform bitwise logical operations, addition, and subtraction. As expected, the addition/subtraction sub-module in the ALU is in the critical path. In order to improve the timing of the design, we can propose two methods.

Since the processor works as a single-cycle module, the first improvement can be applying the pipeline technique to the design. However, pipelining the design will require a lot of modifications to the control unit and other modules. As the code was not written well enough to be easy to modify, and we did not have enough time, we could not progress enough in this direction.

The second proposal is to replace the adder/subtractor module with another IP other than the one currently being used. In the current version, the DW01_addsub IP from Synopsys Design Ware is used. This IP has a control signal that alternates the functionality of the module between adder and subtractor. To improve the timing, we calculated addition and subtraction in parallel using DW01_add and DW01_sub modules and then multiplexed between summation and difference to pass the final result. However, it did not improve the timing and only increased the area usage.

Problem 1.2.5 – Design Metrics (Delay, Area, and Power)

The design met all the timing requirements with a clock of 384.6 MHz, giving a period of 2.6 ns. The critical path of the design was discussed in Problem 1.2.3 and Figures 2 to 6. Additionally, Figure 8 shows the number of paths with the same slack. It can be seen that most of the paths in the design have a slack equal to or greater than 0.004 ns, which stays within the safe margin. However, there are 11 paths that have less slack and are more likely to violate the timing in a more realistic synthesis. Specifically, one of the paths has a slack of exactly 0, which means any slight variation in the physical properties and constraints may lead this path to violate the timing.

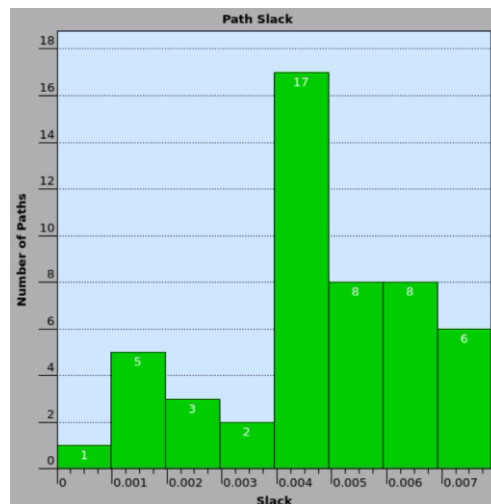


Figure 8. Distribution of paths based on their slack in the design synthesized with FreePDK45

In the case of area, the whole design is taking 40553.78 units, which consists of 17483.30 units for the combinational circuits, 3650.68 units for buffers and inverters, 8424.87 units for non-combinational areas, and 14645.61 units for black box areas such as OpenRAM memory and DesignWare arithmetic unit used in ALU. The complete breakdown of the area for this design is provided in Figure 9.

```
*****
Report : area
Design : Single_Cycle_Top
Version: U-2022.12
Date   : Tue Dec 19 22:25:53 2023
*****

Library(s) Used:

  gsc145nm (File: /home/v60304/mydata/04_syn_freepdk45/pdk_files/gsc145nm.db)
  SRAM_32x64_1rw_tt_1p1v_25c_lib (File: /home/v60304/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_1rw_tt_1p1v_25c.db)

Number of ports:      1223
Number of nets:      9495
Number of cells:      8418
Number of combinational cells: 7291
Number of sequential cells: 1056
Number of macros/black boxes: 2
Number of buf/inv:    2527
Number of references: 13

Combinational area:    17483.301940
Buf/Inv area:          3650.684636
Noncombinational area: 8424.873413
Macro/Black Box area: 14645.611328
Net Interconnect area: undefined (No wire load specified)

Total cell area:       40553.786682
Total area:            undefined
1
```

Figure 9. Area report for the design synthesized using FreePDK45

Furthermore, Figure 10 shows the area report of the tool, indicating the power usage for each category of I/O pads, memories, registers, clock network, and more. The whole design has a power usage of 5.72 mW, among which most of it, 67.69%, is used by just the clock network. Interestingly, the whole combination logic of the design uses nearly the same amount of power as the power used by only memory elements.

```

*****
Report : power
        -analysis_effort low
Design : Single_Cycle_Top
Version: U-2022.12
Date   : Tue Dec 19 22:30:14 2023
*****
Library(s) Used:

  gsc145nm (File: /home/v60304/mydata/04_syn_freepdk45/pdk_files/gsc145nm.db)
  SRAM_32x64_lrw_TT_1p1V_25C_lib (File: /home/v60304/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_lrw_TT_1p1V_25C.db)

Operating Conditions: typical  Library: gsc145nm
Wire Load Model Mode: top

Global Operating Voltage = 1.1
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000pf
  Time Units = 1ns
  Dynamic Power Units = 1mW (derived from V,C,T units)
  Leakage Power Units = 1mW

Attributes
-----
i - Including register clock pin internal power

Cell Internal Power = 5.1977 mW (93%)
Net Switching Power = 409.1378 uW (7%)
-----
Total Dynamic Power = 5.6068 mW (100%)
Cell Leakage Power  = 119.9472 uW

Power Group      Internal      Switching      Leakage      Total
                  Power        Power          Power        Power ( % ) Attrs
-----
io_pad           0.0000          0.0000         0.0000       0.0000 ( 0.00%)
memory           0.8432          1.4264e-02     4.5160e-03   0.8620 ( 15.05%)
black_box        0.0000          0.0000         0.0000       0.0000 ( 0.00%)
clock_network    3.8765          0.0000         0.0000       3.8765 ( 67.69%) i
register         8.5537e-03      5.7659e-03     5.3711e-02   6.8028e-02 ( 1.19%)
sequential       0.0000          0.0000         0.0000       0.0000 ( 0.00%)
combinational    0.4695          0.3891         6.1720e-02   0.9203 ( 16.07%)
-----
Total            5.1977 mW      0.4091 mW      0.1199 mW    5.7268 mW
  
```

Figure 10. Power report for the design synthesized using FreePDK45

Problem 1.3 – Post-synthesis Parasitic (SDF) back-annotation (Gate-level Simulation)

Run a post-synthesis simulation on your RISC-V design using a multiplication program.

In order to run the post-synthesis simulation on our design, we will need the following files:

- Testbench with timing and delays included using SDF_annotate as follows:

```
$sdf_annotate("Single_Cycle_Top_syn.sdf", Single_Cycle_Top);
```

- Post-synthesis Verilog netlist, generated by Synopsys Design Compiler
- FreePDK45 primitive definitions in Verilog
- Generated SRAM primitive definitions in Verilog

For this part, we will be using the Synopsys VCS simulator. As usual, we first source the tool and invoke it with the required arguments as shown below:

```
# source /CMC/scripts/synopsys.vcs_verdi.2022.06-SP2.csh
vcs -kdb -full64 -debug_access -r +v2k +neg_tchk ./rtl/sim_1/Single_Cycle_Top_Tb.v
Single_Cycle_Top_syn.v ./pdk_files/gscl45nm.v ./pdk_files/SRAM_32x64_1rw.v
```

After that, we can open the graphical user interface for viewing the waveform by invoking the following command:

```
./simv -gui
```

Figure 11 shows the waveform monitoring the signals inside the program counter module. As you can see, delays have been considered as the PC_Next register gets updated with some delay after the negative edge of the clock. However, despite no violations during synthesis, the program counter stops working after one clock cycle. Unfortunately, we could not solve this issue in a timely manner.

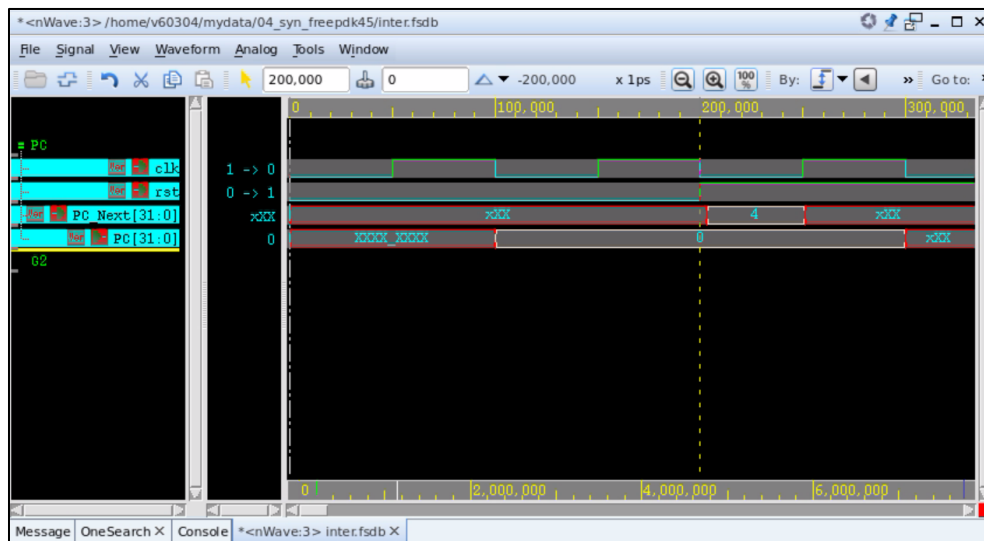


Figure 11. FreePDK45-based post-synthesis simulation waveform, monitoring the Program Counter module

Problem 1.4 – Physical Synthesis using Cadence Innovus

Walk through the following steps to perform a complete physical synthesis on your RISC-V design.

Step 1 – Import Design

To import the design, we navigate through File à Import and set the properties as shown in Figure 12. The Verilog file is the netlist of the design, which Synopsys Design Compiler generated. Next, the Library Exchange Format (LEF) file holds the technology-specific characterizations of metal layers, vias, etc. Lastly, a Multi-Mode/Multi-Corner (MMMC) setup file must be given to the tool.

While trying to import LEF file for the OpenRAM-generated SRAM, the Cadence Innovus was giving error IMPVL-366, stating that the SRAM module is LEF-defined, but the ports defined in the netlist are not defined in the LEF file. However, the LEF file does include the definition for all of the ports. Therefore, it seems there is another issue with the LEF file, which is that the tool is not catching correctly. In order to continue working on the project, we removed the SRAM generated by OpenRAM and replaced it with a custom HDL-coded SRAM module. It is worth noting that using coded SRAM instead of OpenRAM IP tripled the area used by the cell and lowered the frequency from 385 MHz to 285 MHz.

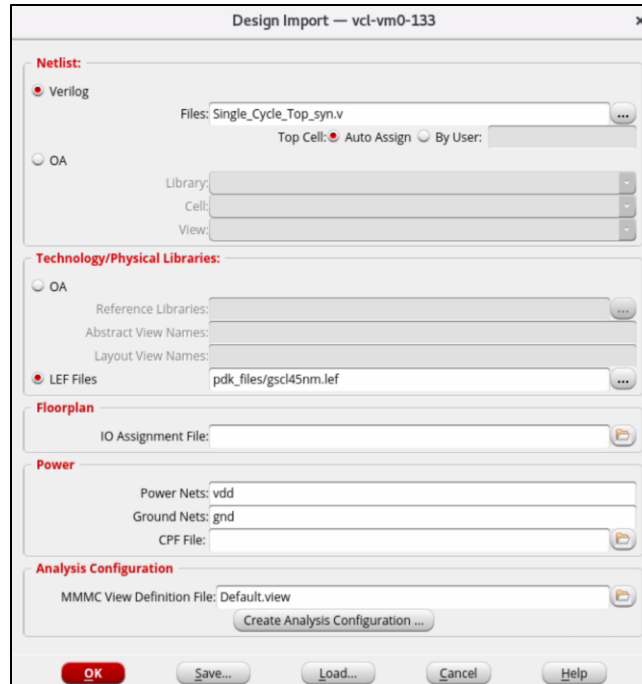


Figure 12. Importing Verilog netlist and library files to Cadence Innovus

The MMMC file was generated using the “Create Analysis Configuration...” dialogue in the tool. Configurations for this file are shown in the figure below.

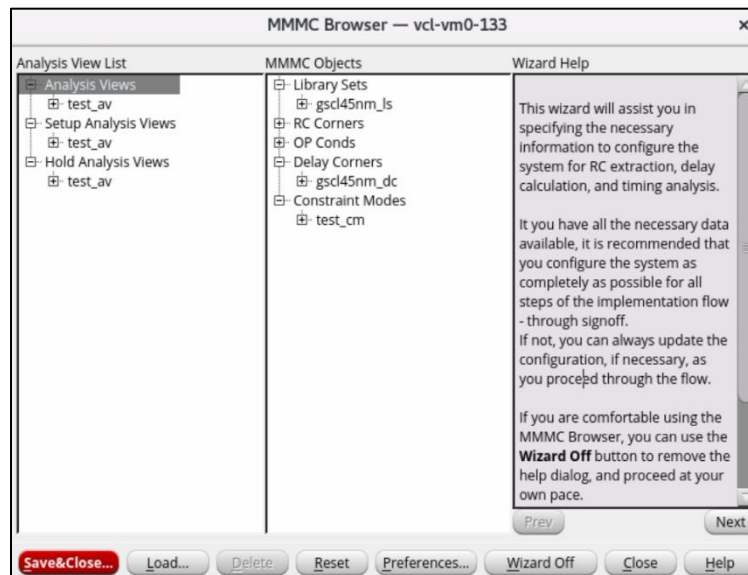


Figure 13. MMMC File configuration for importing FreePDK45 to Cadence Innovus

Step 2 – Connect Global Nets

Global nets in our design are power and ground lines, which are named “vdd” and “gnd”, respectively. To define these global nets in Cadence Innovus, we go through Power à Connect Global Nets and add nets as shown in Figure 14.

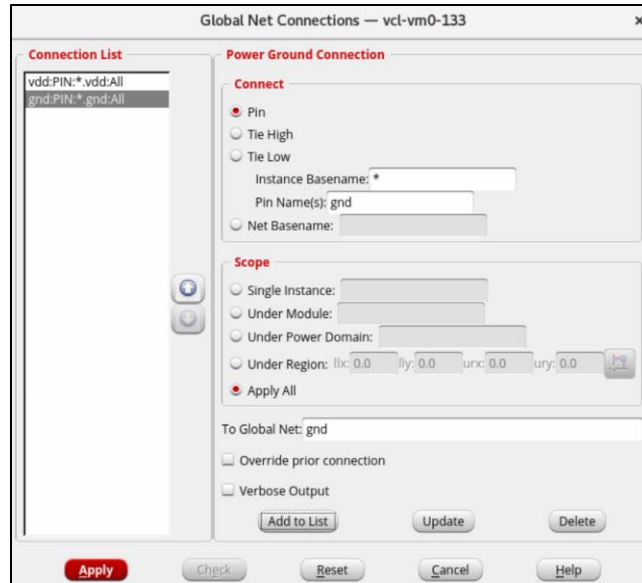


Figure 14. Defining and connecting global nets

Step 3 – Specify Floorplan

Once we have imported our design and defined the global nets, we have to specify the physical area of the tool to be used for the actual placement and routing process in the later stage. To do so, we open the “Floorplan” dropdown menu and select “Specify Floorplan”. This will open the following windows. Change the parameters according to the figure and press OK.

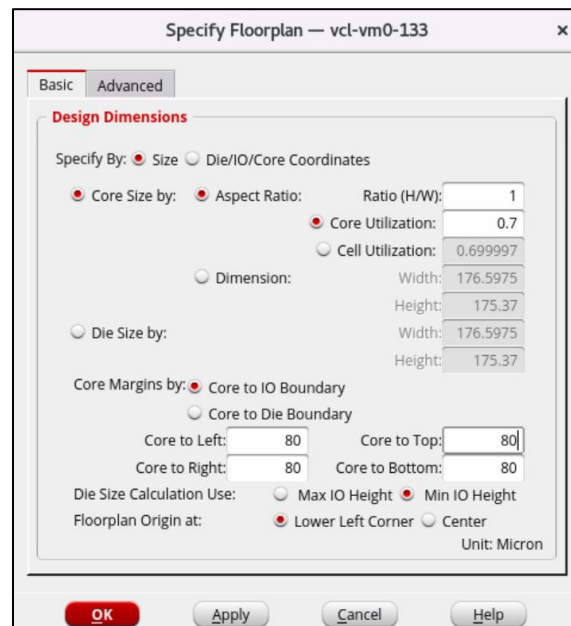


Figure 15. Defining and connecting global nets

After defining the floorplan, we can see in the tool that an area is defined on the die as the core area. The smaller rectangle in the figure below shows this in our design.

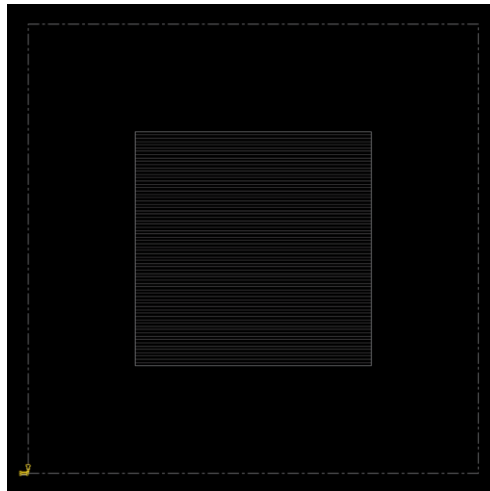


Figure 16. Core area defined for the RISC-V design

Step 4 – Add Power Rings and Power Stripes

We can define power rings through Power à Power Planning à Add Rings dialogue. The setup for the rings defined in this project is shown below.

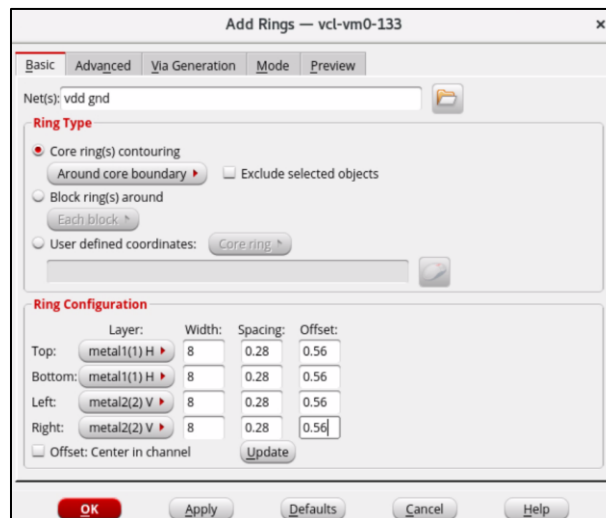


Figure 17. Power ring configuration

We can verify that the power rings are successfully placed over the boundaries of the core by looking at the schematic.

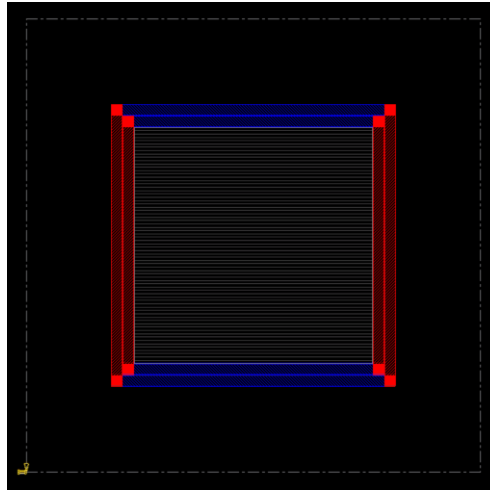


Figure 18. RISC-V die with power rings defined

Furthermore, we have to define power strips for both vdd and gnd nets by opening Power à Power Planning à Add Stripes. The configuration of the added rings can be seen in the figure below.

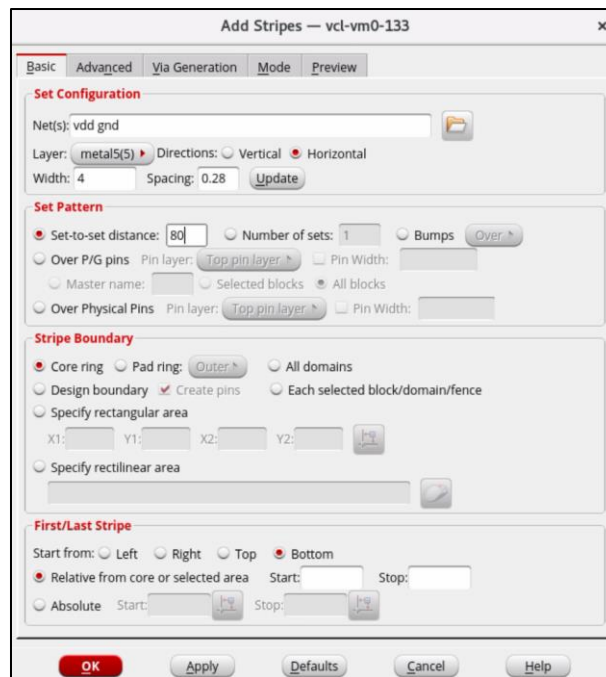


Figure 19. Power stripes configuration

Figure 20 shows the die schematic after adding the power stripes to the design.

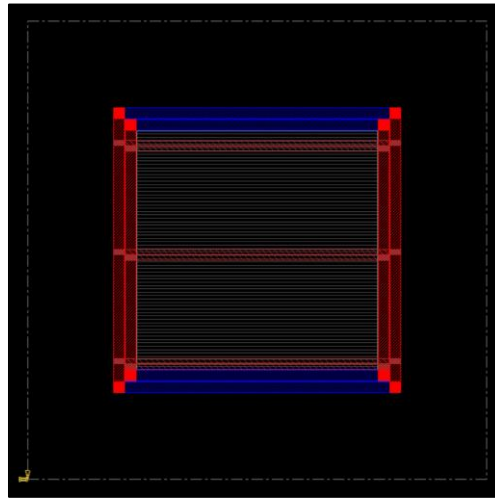


Figure 20. RISC-V die with power rings and stripes defined

Step 5 –Placement

We will perform the placement operation using TCL commands provided to us:

```
createBasicPathGroups -expanded  
get_path_groups  
place_opt_design
```

The placement process will take around 30 minutes to complete. After that, we will see in the schematic that our design cells are placed in the die, as shown in Figure 21.

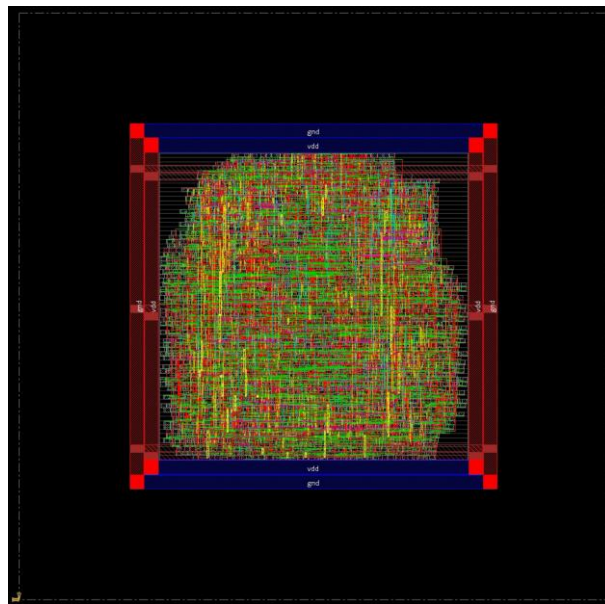


Figure 21. Post-placement and routing RISC-V design on the die

Step 6 –Pre-CTS Timing Analysis and Optimization

We can navigate through Timing → Report Timing to check the post-implementation timing. At this stage, we will generate the Pre-CTS results. Running this command will create a folder named “timingReports”, which is available at “./part_1/04_syn_freepdk45/timingReports_preCts/”.

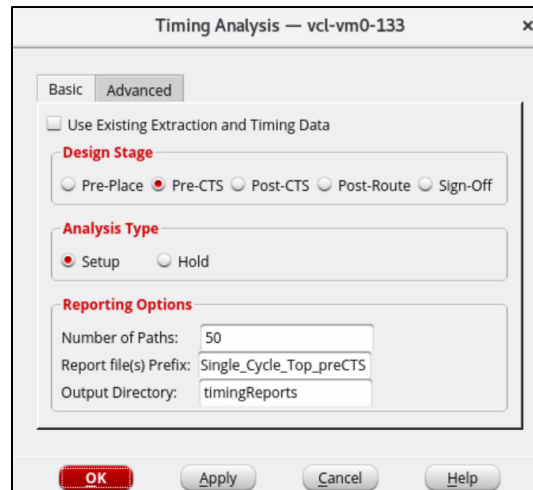


Figure 22. Generating pre-CTS timing analysis report

The report of this stage is shown in Figure 23. As you can see, there are 199 paths that are violating the setup time requirement, with a Worst Negative Slack (WNS) of -0.439 and a Total Negative Slack (TNS) of -14.459. Therefore, it is required to run pre-CTS timing optimization on the design.

timeDesign Summary						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	-0.439	0.562	-0.439	N/A	N/A	0.000
TNS (ns):	-14.457	0.000	-14.457	N/A	N/A	0.000
Violating Paths:	199	0	199	N/A	N/A	0
All Paths:	1032	1032	1032	N/A	N/A	0

DRVs	Real		Total	
	Nr nets (terms)	Worst Vio	Nr nets (terms)	
max_cap	2036 (2036)	-0.309	2036 (2036)	
max_tran	0 (0)	0.000	0 (0)	
max_fanout	0 (0)	0	0 (0)	
max_length	0 (0)	0	0 (0)	

Density: 69.482%
Routing Overflow: 0.00% H and 0.00% V

Figure 23. Pre-CTS setup timing analysis report

To run timing optimization, we can open the windows shown in Figure 24 by opening ECO à Optimize Design. Configurations for running the optimization can be seen in the figure below.



Figure 24. Pre-CTS optimization window

After running the optimization, although the design did not fully meet the timing requirements, it improved much over the previous stage. The post-optimization results are shown in Figure 25. The complete reports are also available in the directory:

“./part_1/04_syn_freepdk45/timingReports_preCts_postOptimization/”.

timeDesign Summary						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	-0.304	0.667	-0.304	N/A	N/A	0.000
TNS (ns):	-5.191	0.000	-5.191	N/A	N/A	0.000
Violating Paths:	103	0	103	N/A	N/A	0
All Paths:	1032	1032	1032	N/A	N/A	0

DRVs	Real		Total	
	Nr nets (terms)	Worst Vio	Nr nets (terms)	
max_cap	2132 (2132)	-0.278	2132 (2132)	
max_tran	0 (0)	0.000	0 (0)	
max_fanout	0 (0)	0	0 (0)	
max_length	0 (0)	0	0 (0)	

Density: 70.474%
Routing Overflow: 0.00% H and 0.00% V

Figure 25. Pre-CTS post-optimization setup timing analysis report

Step 7 – Clock Tree Synthesis (CTS)

Before performing the Clock Tree Synthesis (CTS), clock latency, uncertainty, and ideality must be commented in the SDC file (if any) and sourced again in the tool by invoking the following command:

```
update_constraint_mode -name test_cm -sdc_files Single_Cycle_Top_syn.sdc
```

After we source the new SDC file, we can run the following commands to create a clock tree specification and configure CCOpt library cells.

```
create_ccopt_clock_tree_spec -file ccopt.spec
source ccopt.spec
```

Step 8 – Post-CTS Timing Analysis

Similar to step 6, we can open Optimize windows in the ECO dropdown menu and run the optimization. However, in the windows shown in Figure 24, we change the Design Stage to Post-CTS to see how clock tree synthesis has affected the timing. The timing summary is provided in Figure 26. However, it still has 94 paths, violating the setup time requirement. Full reports are provided in the following path: “./part_1/04_syn_freepdk45/timingReports_postCTS/”.

timeDesign Summary						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	-0.304	0.662	-0.304	N/A	N/A	0.000
TNS (ns):	-4.977	0.000	-4.977	N/A	N/A	0.000
Violating Paths:	94	0	94	N/A	N/A	0
All Paths:	1032	1032	1032	N/A	N/A	0

DRVs	Real		Total	
	Nr nets (terms)	Worst Vio	Nr nets (terms)	
max_cap	2132 (2132)	-0.278	2132 (2132)	
max_tran	0 (0)	0.000	0 (0)	
max_fanout	0 (0)	0	0 (0)	
max_length	0 (0)	0	0 (0)	

Density: 70.474%
Routing Overflow: 0.00% H and 0.00% V

Figure 26. Post-CTS setup timing analysis report

Step 9 – NanoRoute

We can navigate through Route à NanoRoute à Route and configure the routing process as shown in Figure 27.



Figure 27. NanoRoute configuration window

Step 10 – Post-Routing Timing Analysis (Report Critical Path and Maximum Frequency)

Step 10 is also identical to steps 6 and 8. The only difference is that the following command must run in Innovus first in order to perform post-routing timing analysis.

```
setAnalysisMode -analysisType onChipVariation -cpr both
```

Afterward, we can generate the timing report as shown before, but with the Design Stage set to Post-Route. Figure 28 shows the summary of the setup timing analysis generated after NanoRoute. It can be seen that the number of violated paths increased drastically. Based on the slack amount, we expect the design's maximum operating frequency will be around 250 MHz. The complete output of this run is provided in “./part_1/04_syn_freepdk45/timingReports_postRoute_freepdk45”.

timeDesign Summary						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	-0.799	0.606	-0.799	N/A	N/A	0.000
TNS (ns):	-389.581	0.000	-389.581	N/A	N/A	0.000
Violating Paths:	1032	0	1032	N/A	N/A	0
All Paths:	1032	1032	1032	N/A	N/A	0

DRVs	Real		Total	
	Nr nets (terms)	Worst Vio	Nr nets (terms)	
max_cap	2132 (2132)	-0.292	2132 (2132)	
max_tran	0 (0)	0.000	0 (0)	
max_fanout	0 (0)	0	0 (0)	
max_length	0 (0)	0	0 (0)	

Density: 70.474%

Figure 28. Post-route setup timing analysis report

Step 11 – Output Data

The most important file to export from the tool is the netlist. In order to generate the netlist, we can navigate to Design → Save → Netlist. After that, we export the SDF file by running the following commands:

```
setAnalysisMode -analysisType bcwc

write_sdf -max_view test_av \ -min_view test_av \ -edges noedge \ -splitsetuphold \ -
remashold \ -splitrecrem \ -min_period_edges none \ CHIP.sdf
```

The next file is the GDS stick diagram, which is generated by running the commands below. Figure 29 shows the generated schematic, opened by KLayout.

```
setStreamOutMode -specifyViaName default \ -Sevianames false \ -virtualConnection false \
-uniquifyCellNamesPrefix false \ -snapToMGrid false \ -textSize 1 \ -version 3

streamOut CHIP.gds -mapFile streamOut.map \ -stripes 1 -units 1000 -mode ALL
```

These implementation output files are available in “./part_1/04_syn_freepdk45/implementation_outputs”.

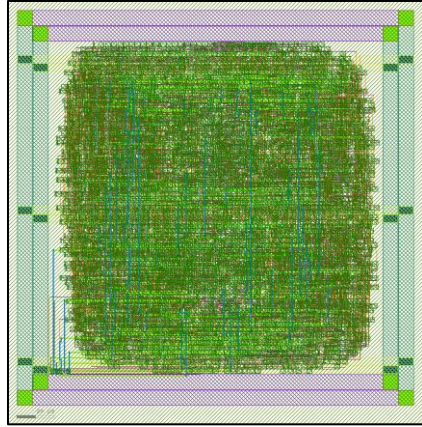


Figure 29. GDS file for the RISC-V design implemented using FreePDK45

Step 12 – Design Metrics (Area and Power)

The total area occupied by this design is 21838.406 units. We can see a detailed breakdown of the area usage by running the command “report_timing”, shown in Figure 30. We can also pass the “-detail” argument to have the area usage for each module categorized by the used element, such as flops, latches, inverters, buffers, etc.

Hinst Name	Module Name	Inst Count	Total Area
Single_Cycle_Top		7298	21838.406
ALU	ALU	1158	2433.790
ALU/DW01_addsub_inst	ALU_DW01_addsub_0	803	1742.042
Control_Unit_Top	Control_Unit_Top	40	80.250
Control_Unit_Top/ALU_Decoder	ALU_Decoder	13	24.873
Control_Unit_Top/Main_Decoder	Main_Decoder	27	55.377
Data_Memory	SRAM_32x64_1rw	0	0.000
Instruction_Memory	SRAM_32x64_1rw	0	0.000
Mux_DataMemory_to_Register	Mux_0	35	114.040
Mux_PCNext	Mux_2	27	55.847
Mux_Register_to_ALU	Mux_1	72	174.580
PC	PC_Module	16	75.557
PC_Adder_4	PC_Adder_1	7	57.724
PC_Adder_4/add_35	PC_Adder_1_DW01_add_0_DW01_add_1	7	57.724
PC_Adder_Imm	PC_Adder_0	9	68.987
PC_Adder_Imm/add_35	PC_Adder_0_DW01_add_0	9	68.987
Register_File	Register_File	5900	18707.237
Sign_Extend	Sign_Extend	20	45.053

Figure 30. Post-implementation area usage report

Furthermore, we can check the power usage of the design by invoking “report_power”. The output of this command is shown in Figure 31. Based on this report, the design uses a total of 9.69 mW. Interestingly, both sequential and combinational logics have roughly the same share in this total power usage.

Total Power						

Total Internal Power:	6.78104081		69.9668%			
Total Switching Power:	2.79957509		28.8860%			
Total Leakage Power:	0.11118232		1.1472%			
Total Power:	9.69179822					

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)	

Sequential	4.69	0.1449	0.05249	4.888	50.43	
Macro	0	0	0	0	0	
I/O	0	0	0	0	0	
Combinational	2.091	2.655	0.05869	4.804	49.57	
Clock (Combinational)	0	0	0	0	0	
Clock (Sequential)	0	0	0	0	0	

Total	6.781	2.8	0.1112	9.692	100	

Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)

vdd	1.1	6.781	2.8	0.1112	9.692	100

* Power Distribution Summary:						
* Highest Average Power: Register File/FE_0FC50_Result_8 (CLKBUF1):						0.02448
* Highest Leakage Power: Register File/FE_0CPC220_Result_31 (CLKBUF1):						0.0001191
* Total Cap: 6.02614e-11 F						
* Total instances in design: 7298						
* Total instances in design with no power: 0						
* Total instances in design with no activity: 0						
* Total Fillers and Decap: 0						

Figure 31. Post-implementation power consumption report

Problem 1.5 – Post-routing Parasitic (SDF) back-annotation (Gate-level Simulation)

Run a post-routing simulation on your RISC-V design using a multiplication program.

Running the post-implementation simulation is quite identical to the post-synthesis one, as explained in Problem 1.3. The only difference is that we have to pass the post-implementation Verilog netlist generated by the Cadence Innovus instead of the post-synthesis netlist that Synopsys Design Compiler generated. Figure 32 shows the waveform window monitoring the Program Counter module. Unfortunately, the design is not simulated correctly, and we could not find the source of the issue.

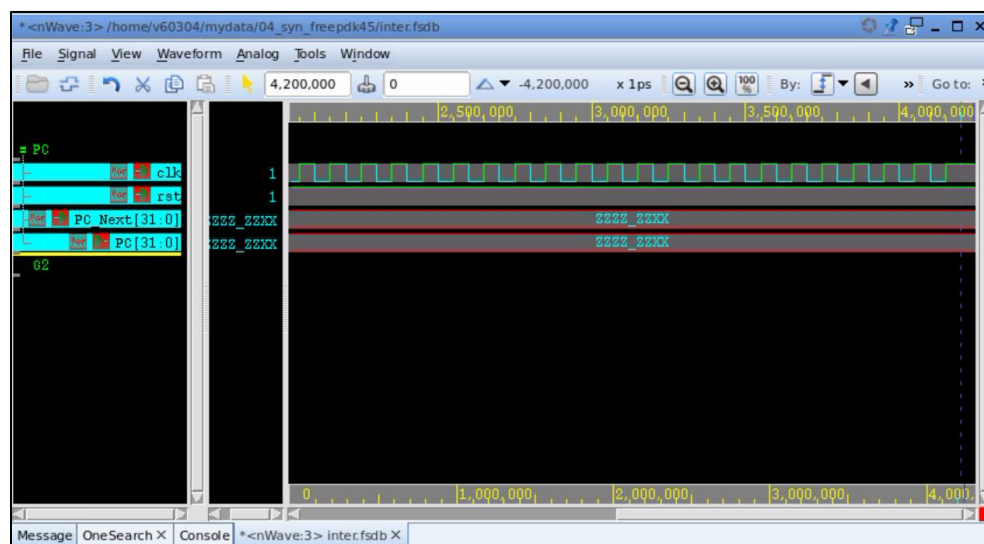


Figure 32. FreePDK45-based post-implementation simulation waveform, monitoring the Program Counter module

Part 2: Synthesis with SkyWater SKY130 Library

Problem 2.1 – SRAM Memory Compiler

Use the OpenRAM SRAM compiler to generate SRAM blocks for your RISC-V design. If the generated file format is “.lib”, convert it to “.db” format using Synopsys Library Compiler.

We reused the same SRAM blocks from Part 1 for this experiment.

Problem 2.2.1 – Gate Synthesis using Synopsys Design Compiler

Walk through the following steps to perform a complete gate synthesis on your RISC-V design.

We got the SkyWater PDK libraries from the “mflowgen/skywater-130nm” repository provided to us. From this repository, we use the “stdcells.db” file, which contains the primitives that will be used in the generated netlist. In our script, we read the exact same Verilog sources that we used in Part 1. Initially, we set all of the design constraints to the same values as well, but we found the resulting synthesis did not meet the design constraints:

Constraint	Cost
-----	-----
min_capacitance	0.00 (MET)
max_transition	0.00 (MET)
max_fanout	0.00 (MET)
max_capacitance	0.00 (MET)
max_delay/setup	1.67 (VIOLATED)
sequential_clock_pulse_width	0.00 (MET)
critical_range	10.00 (VIOLATED)
min_period	0.00 (MET)
max_area	25657.88 (VIOLATED)

Figure 33. SkyWater SKY130-based post-synthesis violations with old constraints

By using SkyWater, the larger 130nm technology results in timing and area violations. To fix this, we increase the maximum area from 500000 to 2500000 and the clock period from 2.6 to 6. After running the script with these changes, the design constraints were all met:

Constraint	Cost
-----	-----
min_capacitance	0.00 (MET)
max_transition	0.00 (MET)
max_fanout	0.00 (MET)
max_capacitance	0.00 (MET)
max_delay/setup	0.00 (MET)
sequential_clock_pulse_width	0.00 (MET)
critical_range	0.00 (MET)
min_period	0.00 (MET)
max_area	0.00 (MET)

Figure 34. SkyWater SKY130-based post-synthesis metrics meeting the constraints

Problem 2.2.2 – False Paths and Multi-cycle Paths

As the design itself remains unchanged, and the only difference from Problem 1 is the PDK used, the false path and multi-cycle path study are the same as explained in the previous problem.

Problem 2.2.3 – Critical Path

Using the same command as before (report_timing -delay_type max -max_path 5), we get these five paths:

Startpoint: Instruction_Memory/clk0 (internal path startpoint clocked by clk)		
Endpoint: Data_Memory (rising edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
Single_Cycle_Top	Small	sky130_fd_sc_hd__tt_025C_1v80
Point	Incr	Path
clock clk (fall edge)	3.00	3.00
clock network delay (ideal)	0.01	3.01
input external delay	0.00	3.01 f
Instruction_Memory/clk0 (SRAM_32x64_1rw)	0.00 #	3.01 f
Instruction_Memory/dout0[20] (SRAM_32x64_1rw)	0.32	3.33 r
Register_File/A2[0] (Register_File)	0.00	3.33 r
Register_File/U29/Y (sky130_fd_sc_hd__nor2b_1)	0.23	3.56 r
Register_File/U81/Y (sky130_fd_sc_hd__nand2_1)	0.17	3.73 f
Register_File/U80/Y (sky130_fd_sc_hd__inv_2)	0.29	4.02 r
Register_File/U1690/Y (sky130_fd_sc_hd__a22oi_1)	0.14	4.16 f
Register_File/U1686/Y (sky130_fd_sc_hd__nand4_1)	0.09	4.25 r
Register_File/U1680/Y (sky130_fd_sc_hd__nor2_1)	0.05	4.30 f
Register_File/U1679/Y (sky130_fd_sc_hd__mux2i_1)	0.11	4.41 r
Register_File/U200/Y (sky130_fd_sc_hd__nor2b_1)	0.14	4.55 r
Register_File/RD2[22] (Register_File)	0.00	4.55 r
Mux_Register_to_ALU/a[22] (Mux_1)	0.00	4.55 r
Mux_Register_to_ALU/U8/X (sky130_fd_sc_hd__mux2_2)	0.17	4.72 r
Mux_Register_to_ALU/c[22] (Mux_1)	0.00	4.72 r
ALU/B[22] (ALU)	0.00	4.72 r
ALU/DW01_addsub_inst/B[22] (ALU_DW01_addsub_2)	0.00	4.72 r
ALU/DW01_addsub_inst/U604/Y (sky130_fd_sc_hd__clkinv_1)	0.06	4.78 f
ALU/DW01_addsub_inst/U601/Y (sky130_fd_sc_hd__nand2_1)	0.06	4.84 r
ALU/DW01_addsub_inst/U445/Y (sky130_fd_sc_hd__nand2_1)	0.07	4.91 f
ALU/DW01_addsub_inst/U450/Y (sky130_fd_sc_hd__nor2_2)	0.17	5.08 r
ALU/DW01_addsub_inst/U605/Y (sky130_fd_sc_hd__o21ai_1)	0.10	5.18 f
ALU/DW01_addsub_inst/U581/Y (sky130_fd_sc_hd__a21oi_2)	0.12	5.30 r
ALU/DW01_addsub_inst/U549/Y (sky130_fd_sc_hd__o21ai_1)	0.09	5.39 f
ALU/DW01_addsub_inst/U14/Y (sky130_fd_sc_hd__a21oi_1)	0.12	5.51 r
ALU/DW01_addsub_inst/U628/Y (sky130_fd_sc_hd__o21ai_1)	0.08	5.58 f
ALU/DW01_addsub_inst/U718/Y (sky130_fd_sc_hd__inv_1)	0.08	5.67 r
ALU/DW01_addsub_inst/U486/Y (sky130_fd_sc_hd__inv_1)	0.04	5.71 f
ALU/DW01_addsub_inst/U484/Y (sky130_fd_sc_hd__nand2_1)	0.07	5.78 r
ALU/DW01_addsub_inst/U425/Y (sky130_fd_sc_hd__nand2_2)	0.08	5.86 f
ALU/DW01_addsub_inst/SUM[31] (ALU_DW01_addsub_2)	0.00	5.86 f
ALU/U31/Y (sky130_fd_sc_hd__nand4bb_1)	0.09	5.95 r
ALU/U39/Y (sky130_fd_sc_hd__nand2_1)	0.05	6.00 f
ALU/Result[0] (ALU)	0.00	6.00 f
Data_Memory/addr0[0] (SRAM_32x64_1rw)	0.00	6.00 f
data arrival time		6.00
clock clk (rise edge)	6.00	6.00
clock network delay (ideal)	0.01	6.01
Data_Memory/clk0 (SRAM_32x64_1rw)	0.00	6.01 r
library setup time	-0.01	6.00
data required time		6.00
data required time		6.00
data arrival time		-6.00
slack (MET)		0.00

Figure 35. 1st critical path in the design synthesized using SKY130 with a clock period of 6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[0]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
Single_Cycle_Top	Small	sky130_fd_sc_hd_tt_025C_1v80
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 f
rst (in)	0.00	0.03 f
Register_File/rst (Register_File)	0.00	0.03 f
Register_File/U160/Y (sky130_fd_sc_hd_inv_1)	0.03	0.06 r
Register_File/U158/Y (sky130_fd_sc_hd_nor2_1)	0.04	0.10 f
Register_File/RD2[6] (Register_File)	0.00	0.10 f
Mux_Register_to_ALU/a[6] (Mux_1)	0.00	0.10 f
Mux_Register_to_ALU/U45/X (sky130_fd_sc_hd_mux2_4)	0.27	0.37 f
Mux_Register_to_ALU/c[6] (Mux_1)	0.00	0.37 f
ALU/B[6] (ALU)	0.00	0.37 f
ALU/DW01_addsub_inst/B[6] (ALU_DW01_addsub_2)	0.00	0.37 f
ALU/DW01_addsub_inst/U642/Y (sky130_fd_sc_hd_inv_1)	0.06	0.42 r
ALU/DW01_addsub_inst/U442/Y (sky130_fd_sc_hd_nand2_1)	0.07	0.49 f
ALU/DW01_addsub_inst/U548/Y (sky130_fd_sc_hd_nand2_2)	0.08	0.57 r
ALU/DW01_addsub_inst/U537/Y (sky130_fd_sc_hd_nor2_2)	0.04	0.61 f
ALU/DW01_addsub_inst/U729/Y (sky130_fd_sc_hd_nor2_2)	0.12	0.73 r
ALU/DW01_addsub_inst/U313/Y (sky130_fd_sc_hd_nand2_1)	0.08	0.81 f
ALU/DW01_addsub_inst/U741/Y (sky130_fd_sc_hd_o21ai_2)	0.17	0.98 r
ALU/DW01_addsub_inst/U701/Y (sky130_fd_sc_hd_inv_1)	0.13	1.11 f
ALU/DW01_addsub_inst/U257/Y (sky130_fd_sc_hd_o21ai_1)	0.16	1.27 r
ALU/DW01_addsub_inst/U242/Y (sky130_fd_sc_hd_xnor2_1)	0.09	1.36 f
ALU/DW01_addsub_inst/SUM[12] (ALU_DW01_addsub_2)	0.00	1.36 f
ALU/U171/Y (sky130_fd_sc_hd_nand2_1)	0.06	1.41 r
ALU/U172/Y (sky130_fd_sc_hd_o31lai_1)	0.10	1.52 f
ALU/U248/Y (sky130_fd_sc_hd_nor4_1)	0.29	1.80 r
ALU/U250/Y (sky130_fd_sc_hd_nand4_1)	0.11	1.92 f
ALU/U252/Y (sky130_fd_sc_hd_nor2_1)	0.14	2.06 r
ALU/Zero (ALU)	0.00	2.06 r
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/U9/Y (sky130_fd_sc_hd_xnor2_1)	0.09	2.15 f
Control_Unit_Top/Main_Decoder/U8/Y (sky130_fd_sc_hd_nor2_2)	0.10	2.25 r
Control_Unit_Top/Main_Decoder/U10/X (sky130_fd_sc_hd_buf_6)	0.15	2.40 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	2.40 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	2.40 r
Mux_PCNext/s (Mux_2)	0.00	2.40 r
Mux_PCNext/U2/X (sky130_fd_sc_hd_mux2_1)	0.31	2.72 f
Mux_PCNext/c[0] (Mux_2)	0.00	2.72 f
PC/PC_Next[0] (PC_Module)	0.00	2.72 f
PC/U28/X (sky130_fd_sc_hd_and2_0)	0.13	2.85 f
PC/PC_reg[0]/D (sky130_fd_sc_hd_dfrtn_1)	0.00	2.85 f
data arrival time		2.85
clock clk (fall edge)	3.00	3.00
clock network delay (ideal)	0.01	3.01
PC/PC_reg[0]/CLK_N (sky130_fd_sc_hd_dfrtn_1)	0.00	3.01 f
library setup time	-0.16	2.85
data required time		2.85
data required time		2.85
data arrival time		-2.85
slack (MEI)		0.00

Figure 36. 2nd critical path in the design synthesized using SKY130 with a clock period of 6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[1]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
Single_Cycle_Top	Small	sky130_fd_sc_hd__tt_025C_1v80
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 f
rst (in)	0.00	0.03 f
Register_File/rst (Register_File)	0.00	0.03 f
Register_File/U160/Y (sky130_fd_sc_hd__inv_1)	0.03	0.06 r
Register_File/U158/Y (sky130_fd_sc_hd__nor2_1)	0.04	0.10 f
Register_File/RD2[6] (Register_File)	0.00	0.10 f
Mux_Register_to_ALU/a[6] (Mux_1)	0.00	0.10 f
Mux_Register_to_ALU/U45/X (sky130_fd_sc_hd__mux2_4)	0.27	0.37 f
Mux_Register_to_ALU/c[6] (Mux_1)	0.00	0.37 f
ALU/B[6] (ALU)	0.00	0.37 f
ALU/DW01_addsub_inst/B[6] (ALU_DW01_addsub_2)	0.00	0.37 f
ALU/DW01_addsub_inst/U642/Y (sky130_fd_sc_hd__inv_1)	0.06	0.42 r
ALU/DW01_addsub_inst/U442/Y (sky130_fd_sc_hd__nand2_1)	0.07	0.49 f
ALU/DW01_addsub_inst/U548/Y (sky130_fd_sc_hd__nand2_2)	0.08	0.57 r
ALU/DW01_addsub_inst/U537/Y (sky130_fd_sc_hd__nor2_2)	0.04	0.61 f
ALU/DW01_addsub_inst/U729/Y (sky130_fd_sc_hd__nor2_2)	0.12	0.73 r
ALU/DW01_addsub_inst/U313/Y (sky130_fd_sc_hd__nand2_1)	0.08	0.81 f
ALU/DW01_addsub_inst/U741/Y (sky130_fd_sc_hd__o2lai_2)	0.17	0.98 r
ALU/DW01_addsub_inst/U701/Y (sky130_fd_sc_hd__inv_1)	0.13	1.11 f
ALU/DW01_addsub_inst/U257/Y (sky130_fd_sc_hd__o2lai_1)	0.16	1.27 r
ALU/DW01_addsub_inst/U242/Y (sky130_fd_sc_hd__xnor2_1)	0.09	1.36 f
ALU/DW01_addsub_inst/SUM[12] (ALU_DW01_addsub_2)	0.00	1.36 f
ALU/U171/Y (sky130_fd_sc_hd__nand2_1)	0.06	1.41 r
ALU/U172/Y (sky130_fd_sc_hd__o31lai_1)	0.10	1.52 f
ALU/U248/Y (sky130_fd_sc_hd__nor4_1)	0.29	1.80 r
ALU/U250/Y (sky130_fd_sc_hd__nand4_1)	0.11	1.92 f
ALU/U252/Y (sky130_fd_sc_hd__nor2_1)	0.14	2.06 r
ALU/Zero (ALU)	0.00	2.06 r
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/U9/Y (sky130_fd_sc_hd__xnor2_1)	0.09	2.15 f
Control_Unit_Top/Main_Decoder/U8/Y (sky130_fd_sc_hd__nor2_2)	0.10	2.25 r
Control_Unit_Top/Main_Decoder/U10/X (sky130_fd_sc_hd__buf_6)	0.15	2.40 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	2.40 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	2.40 r
Mux_PCNext/s (Mux_2)	0.00	2.40 r
Mux_PCNext/U3/X (sky130_fd_sc_hd__mux2_1)	0.31	2.72 f
Mux_PCNext/c[1] (Mux_2)	0.00	2.72 f
PC/PC_Next[1] (PC_Module)	0.00	2.72 f
PC/U29/X (sky130_fd_sc_hd__and2_0)	0.13	2.85 f
PC/PC_reg[1]/D (sky130_fd_sc_hd__dfrtn_1)	0.00	2.85 f
data arrival time		2.85
clock clk (fall edge)	3.00	3.00
clock network delay (ideal)	0.01	3.01
PC/PC_reg[1]/CLK_N (sky130_fd_sc_hd__dfrtn_1)	0.00	3.01 f
library setup time	-0.16	2.85
data required time		2.85
data required time		2.85
data arrival time		-2.85
slack (MET)		0.00

Figure 37. 3rd critical path in the design synthesized using SKY130 with a clock period of 6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[2]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
Single_Cycle_Top	Small	sky130_fd_sc_hd_tt_025C_1v80
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 f
rst (in)	0.00	0.03 f
Register_File/rst (Register_File)	0.00	0.03 f
Register_File/U160/Y (sky130_fd_sc_hd_inv_1)	0.03	0.06 r
Register_File/U158/Y (sky130_fd_sc_hd_nor2_1)	0.04	0.10 f
Register_File/RD2[6] (Register_File)	0.00	0.10 f
Mux_Register_to_ALU/a[6] (Mux_1)	0.00	0.10 f
Mux_Register_to_ALU/U45/X (sky130_fd_sc_hd_mux2_4)	0.27	0.37 f
Mux_Register_to_ALU/c[6] (Mux_1)	0.00	0.37 f
ALU/B[6] (ALU)	0.00	0.37 f
ALU/DW01_addsub_inst/B[6] (ALU_DW01_addsub_2)	0.00	0.37 f
ALU/DW01_addsub_inst/U642/Y (sky130_fd_sc_hd_inv_1)	0.06	0.42 r
ALU/DW01_addsub_inst/U442/Y (sky130_fd_sc_hd_nand2_1)	0.07	0.49 f
ALU/DW01_addsub_inst/U548/Y (sky130_fd_sc_hd_nand2_2)	0.08	0.57 r
ALU/DW01_addsub_inst/U537/Y (sky130_fd_sc_hd_nor2_2)	0.04	0.61 f
ALU/DW01_addsub_inst/U729/Y (sky130_fd_sc_hd_nor2_2)	0.12	0.73 r
ALU/DW01_addsub_inst/U313/Y (sky130_fd_sc_hd_nand2_1)	0.08	0.81 f
ALU/DW01_addsub_inst/U741/Y (sky130_fd_sc_hd_o2lai_2)	0.17	0.98 r
ALU/DW01_addsub_inst/U701/Y (sky130_fd_sc_hd_inv_1)	0.13	1.11 f
ALU/DW01_addsub_inst/U257/Y (sky130_fd_sc_hd_o2lai_1)	0.16	1.27 r
ALU/DW01_addsub_inst/U242/Y (sky130_fd_sc_hd_xnor2_1)	0.09	1.36 f
ALU/DW01_addsub_inst/SUM[12] (ALU_DW01_addsub_2)	0.00	1.36 f
ALU/U171/Y (sky130_fd_sc_hd_nand2_1)	0.06	1.41 r
ALU/U172/Y (sky130_fd_sc_hd_o31lai_1)	0.10	1.52 f
ALU/U248/Y (sky130_fd_sc_hd_nor4_1)	0.29	1.80 r
ALU/U250/Y (sky130_fd_sc_hd_nand4_1)	0.11	1.92 f
ALU/U252/Y (sky130_fd_sc_hd_nor2_1)	0.14	2.06 r
ALU/Zero (ALU)	0.00	2.06 r
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/U9/Y (sky130_fd_sc_hd_xnor2_1)	0.09	2.15 f
Control_Unit_Top/Main_Decoder/U8/Y (sky130_fd_sc_hd_nor2_2)	0.10	2.25 r
Control_Unit_Top/Main_Decoder/U10/X (sky130_fd_sc_hd_buf_6)	0.15	2.40 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	2.40 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	2.40 r
Mux_PCNext/s (Mux_2)	0.00	2.40 r
Mux_PCNext/U4/X (sky130_fd_sc_hd_mux2_1)	0.31	2.72 f
Mux_PCNext/c[2] (Mux_2)	0.00	2.72 f
PC/PC_Next[2] (PC_Module)	0.00	2.72 f
PC/U30/X (sky130_fd_sc_hd_and2_0)	0.13	2.85 f
PC/PC_reg[2]/D (sky130_fd_sc_hd_dfrtn_1)	0.00	2.85 f
data arrival time		2.85
clock clk (fall edge)	3.00	3.00
clock network delay (ideal)	0.01	3.01
PC/PC_reg[2]/CLK_N (sky130_fd_sc_hd_dfrtn_1)	0.00	3.01 f
library setup time	-0.16	2.85
data required time		2.85
data required time		2.85
data arrival time		-2.85
slack (MET)		0.00

Figure 38. 4th critical path in the design synthesized using SKY130 with a clock period of 6 ns

Startpoint: rst (input port clocked by clk)		
Endpoint: PC/PC_reg[3]		
(falling edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
Single_Cycle_Top	Small	sky130_fd_sc_hd__tt_025C_1v80
Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.01	0.01
input external delay	0.02	0.03 f
rst (in)	0.00	0.03 f
Register_File/rst (Register_File)	0.00	0.03 f
Register_File/U160/Y (sky130_fd_sc_hd__inv_1)	0.03	0.06 r
Register_File/U158/Y (sky130_fd_sc_hd__nor2_1)	0.04	0.10 f
Register_File/RD2[6] (Register_File)	0.00	0.10 f
Mux_Register_to_ALU/a[6] (Mux_1)	0.00	0.10 f
Mux_Register_to_ALU/U45/X (sky130_fd_sc_hd__mux2_4)	0.27	0.37 f
Mux_Register_to_ALU/c[6] (Mux_1)	0.00	0.37 f
ALU/B[6] (ALU)	0.00	0.37 f
ALU/DW01_addsub_inst/B[6] (ALU_DW01_addsub_2)	0.00	0.37 f
ALU/DW01_addsub_inst/U642/Y (sky130_fd_sc_hd__inv_1)	0.06	0.42 r
ALU/DW01_addsub_inst/U442/Y (sky130_fd_sc_hd__nand2_1)	0.07	0.49 f
ALU/DW01_addsub_inst/U548/Y (sky130_fd_sc_hd__nand2_2)	0.08	0.57 r
ALU/DW01_addsub_inst/U537/Y (sky130_fd_sc_hd__nor2_2)	0.04	0.61 f
ALU/DW01_addsub_inst/U729/Y (sky130_fd_sc_hd__nor2_2)	0.12	0.73 r
ALU/DW01_addsub_inst/U313/Y (sky130_fd_sc_hd__nand2_1)	0.08	0.81 f
ALU/DW01_addsub_inst/U741/Y (sky130_fd_sc_hd__o21ai_2)	0.17	0.98 r
ALU/DW01_addsub_inst/U701/Y (sky130_fd_sc_hd__inv_1)	0.13	1.11 f
ALU/DW01_addsub_inst/U257/Y (sky130_fd_sc_hd__o21ai_1)	0.16	1.27 r
ALU/DW01_addsub_inst/U242/Y (sky130_fd_sc_hd__xnor2_1)	0.09	1.36 f
ALU/DW01_addsub_inst/SUM[12] (ALU_DW01_addsub_2)	0.00	1.36 f
ALU/U171/Y (sky130_fd_sc_hd__nand2_1)	0.06	1.41 r
ALU/U172/Y (sky130_fd_sc_hd__o311ai_1)	0.10	1.52 f
ALU/U248/Y (sky130_fd_sc_hd__nor4_1)	0.29	1.80 r
ALU/U250/Y (sky130_fd_sc_hd__nand4_1)	0.11	1.92 f
ALU/U252/Y (sky130_fd_sc_hd__nor2_1)	0.14	2.06 r
ALU/Zero (ALU)	0.00	2.06 r
Control_Unit_Top/Zero (Control_Unit_Top)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/Zero (Main_Decoder)	0.00	2.06 r
Control_Unit_Top/Main_Decoder/U9/Y (sky130_fd_sc_hd__xnor2_1)	0.09	2.15 f
Control_Unit_Top/Main_Decoder/U8/Y (sky130_fd_sc_hd__nor2_2)	0.10	2.25 r
Control_Unit_Top/Main_Decoder/U10/X (sky130_fd_sc_hd__buf_6)	0.15	2.40 r
Control_Unit_Top/Main_Decoder/PCSrc (Main_Decoder)	0.00	2.40 r
Control_Unit_Top/PCSrc (Control_Unit_Top)	0.00	2.40 r
Mux_PCNext/s (Mux_2)	0.00	2.40 r
Mux_PCNext/U5/X (sky130_fd_sc_hd__mux2_1)	0.31	2.72 f
Mux_PCNext/c[3] (Mux_2)	0.00	2.72 f
PC/PC_Next[3] (PC_Module)	0.00	2.72 f
PC/U31/X (sky130_fd_sc_hd__and2_0)	0.13	2.85 f
PC/PC_reg[3]/D (sky130_fd_sc_hd__dfrtn_1)	0.00	2.85 f
data arrival time		2.85
clock clk (fall edge)	3.00	3.00
clock network delay (ideal)	0.01	3.01
PC/PC_reg[3]/CLK_N (sky130_fd_sc_hd__dfrtn_1)	0.00	3.01 f
library setup time	-0.16	2.85
data required time		2.85
data required time		2.85
data arrival time		-2.85
slack (MET)		0.00

Figure 39. 5th critical path in the design synthesized using SKY130 with a clock period of 6 ns

Analysis: The first path shown is the true critical path, just like with Part 1. It takes the route from the instruction memory, where the next instruction is read, to the data memory, where new values may be written by an assembly program. As this is similar to Part 1, refer back to that section for more details.

Problem 2.2.4 – Timing Improvement

Several methods for timing improvement were proposed in Problem 1.2.4. As these modifications are in the RTL level, they will be also applicable with the new SKY130 PDK used.

Problem 2.2.5 – Design Metrics (Delay, Area, and Power)

The synthesized device is slower compared to the design in Problem 1. The reason is that the SkyWater 130 nm technology is larger than FreePDK. In order to address this issue, the clock period was increased from 2.6 ns to 6 ns, which is the same as the critical path delay. This means that the clock frequency has decreased from 384.6MHz to 166.7MHz.

Additionally, the area usage surged by using this 130 nm PDK. The area usage report is shown in the figure below. We can see that the total cell area has almost doubled from about forty-thousand units to 74010 units. Below is the Power report:

```
*****
Report : area
Design : Single_Cycle_Top
Version: U-2022.12
Date   : Thu Dec 21 21:23:14 2023
*****

Library(s) Used:

  skyl130_fd_sc_hd_tt_025C_1v80 (File: /home/v62970/mydata/04_syn_freepdk45/skywater/stdcells.db)
  SRAM_32x64_1rw_TT_1p1V_25C_lib (File: /home/v62970/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_1rw_TT_1p1V_25C.db)

Number of ports:          1126
Number of nets:           7855
Number of cells:          6872
Number of combinational cells: 4999
Number of sequential cells: 1056
Number of macros/black boxes: 2
Number of buf/inv:        1647
Number of references:      15

Combinational area:       32939.090368
Buf/Inv area:             6382.371010
Noncombinational area:    26425.344177
Macro/Black Box area:     14645.611328
Net Interconnect area:    undefined (Wire load has zero net area)

Total cell area:          74010.045873
Total area:               undefined
```

Figure 40. Area report for the design synthesized using SKY130

```

*****
Report : power
        -analysis_effort low
Design : Single_Cycle_Top
Version: U-2022.12
Date   : Thu Dec 21 21:25:54 2023
*****

Library(s) Used:

    sky130_fd_sc_hd__tt_025C_1v80 (File: /home/v62970/mydata/04_syn_freepdk45/skywater/stdcells.db)
    SRAM_32x64_1rw_TT_1p1V_25C_lib (File: /home/v62970/mydata/04_syn_freepdk45/pdk_files/SRAM_32x64_1rw_TT_1p1V_25C.db)

Operating Conditions: tt_025C_1v80 Library: sky130_fd_sc_hd__tt_025C_1v80
Wire Load Model Mode: top

Design      Wire Load Model      Library
-----
Single_Cycle_Top      Small      sky130_fd_sc_hd__tt_025C_1v80

Global Operating Voltage = 1.8
Power-specific unit information :
    Voltage Units = 1V
    Capacitance Units = 1.000000pf
    Time Units = 1ns
    Dynamic Power Units = 1mW (derived from V,C,T units)
    Leakage Power Units = 1nW

Attributes
-----
i - Including register clock pin internal power

Cell Internal Power = 7.7931 mW (97%)
Net Switching Power = 247.9733 uW (3%)
-----
Total Dynamic Power = 8.0410 mW (100%)
Cell Leakage Power = 4.5406 uW

Power Group      Internal Power      Switching Power      Leakage Power      Total Power ( % ) Attrs
-----
io_pad           0.0000           0.0000           0.0000           0.0000 ( 0.00%)
memory          0.3654          1.8038e-02          4.5160e+03          0.3879 ( 4.82%)
black_box       0.0000           0.0000           0.0000           0.0000 ( 0.00%)
clock_network   7.2848           0.0000           0.0000           7.2848 ( 90.54%) i
register        4.6639e-03       1.7607e-03          14.5571           6.4356e-03 ( 0.08%)
sequential      0.0000           0.0000           0.0000           0.0000 ( 0.00%)
combinational   0.1382           0.2282           10.0350           0.3664 ( 4.55%)
-----
Total           7.7931 mW       0.2480 mW          4.5406e+03 nW      8.0456 mW

```

Figure 41. Power report for the design synthesized using SKY130

Problem 2.3 – Post-synthesis Parasitic (SDF) back-annotation (Gate-level Simulation)

Run a post-synthesis simulation on your RISC-V design using a multiplication program.

The TCL script we ran in the previous part includes the write command, which generates the Verilog netlist of the CPU using the SkyWater PDK. Once we generate this, we then try to simulate it using the Synopsys VCS with the SkyWater “primitives.v” and “stdcells.v” files:

```

vcs -kdb -full64 -debug_access -r +v2k +neg_tchk ./rtl/sim_1/Single_Cycle_Top_Tb.v
Single_Cycle_Top_syn.v ./skywater/stdcells.v ./skywater/primitives.v
./pdk_files/SRAM_32x64_1rw.v

```

As in Part 1, once the Verdi GUI launches, we add the Program Counter module to the waveforms and run the simulation. The waveform is shown in the figure below. However, unfortunately, there is an issue with the simulation that causes the PC never to update.

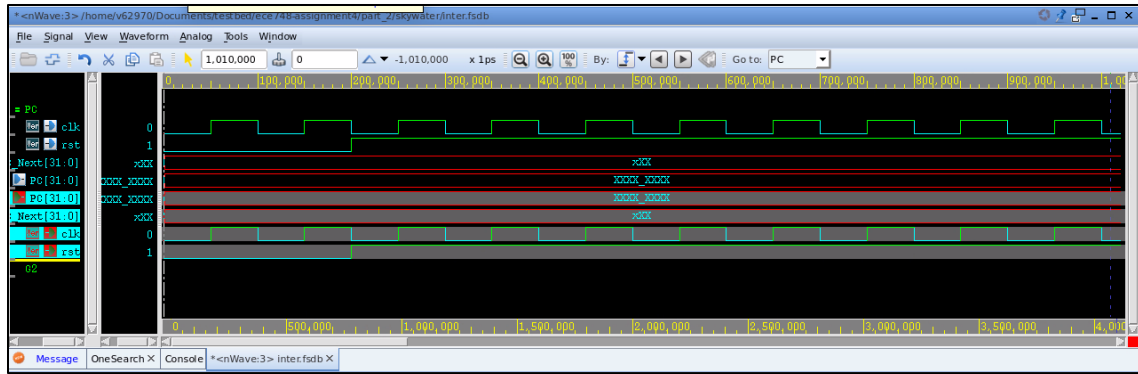


Figure 42. SKY130-based post-synthesis simulation waveform, monitoring the Program Counter module

Problem 2.4 – Physical Synthesis using Cadence Innovus

Walk through the following steps to perform a complete physical synthesis on your RISC-V design.

Step 1 – Import Design

Just as in Part 1, we imported our synthesized Verilog file into Cadence Innovus. Like in Part 1, we were unable to load the SRAM module generated by OpenRAM, and so we had to resynthesize the RISC-V CPU without it. We imported the design similarly to Part 1 as shown below.

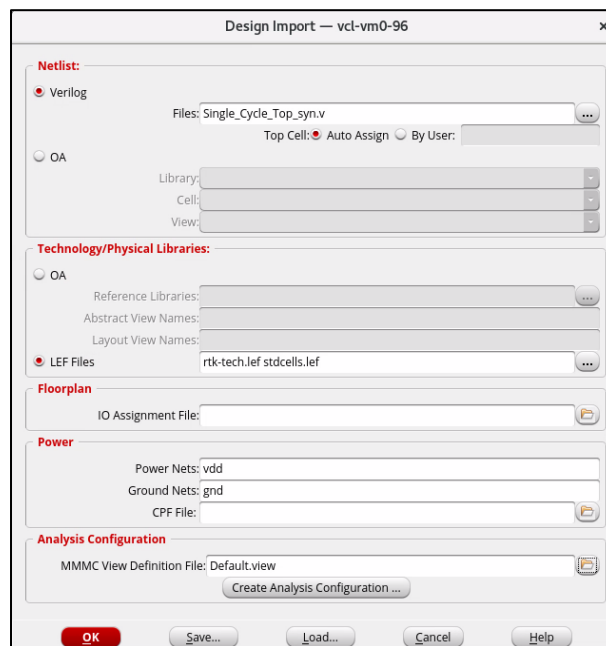


Figure 43. Importing Verilog netlist and library files to Cadence Innovus

For the LEF files, we instead used the “rtk-tech.lef” and “stdcells.lef”, which the SkyWater repository provided. When setting the MMMC file, we used the provided “stdcells.lib” for our timing file because the SkyWater repository did not provide a *.tlf file.

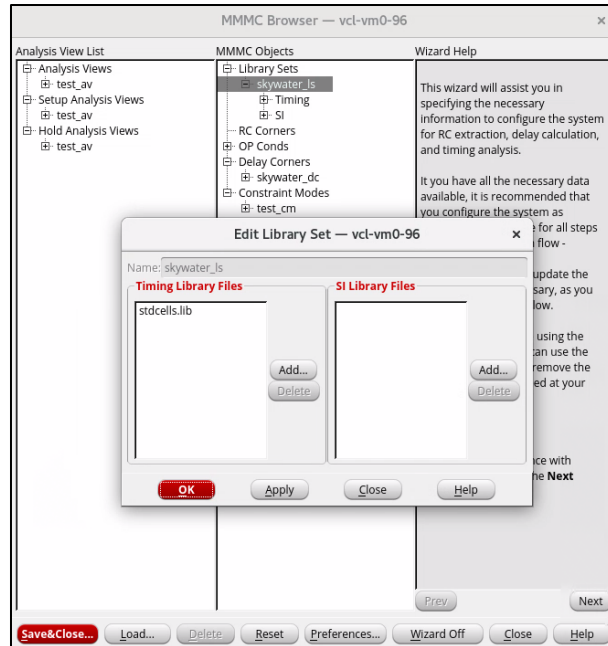


Figure 44. MMMC File configuration for importing SKY130 to Cadence Innovus

Step 2 & 3– Connect Global Nets/Specify Floorplan

The process for these steps is quite identical to what was done in Problem 1. Thus, screenshots are not copied here again.

Step 4 – Add Power Rings and Power Stripes

We specify the floorplan using the exact specifications as Problem 1. After adding the power ring, we get the following schematic:

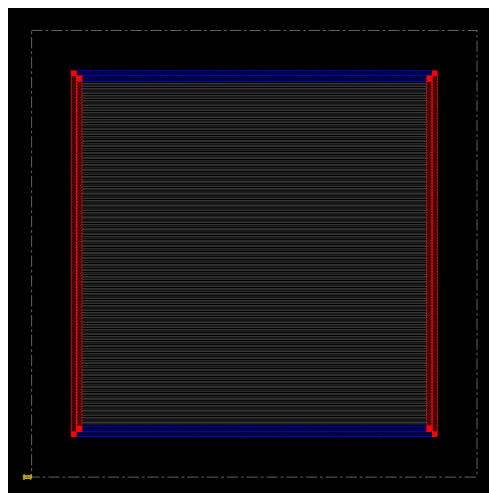


Figure 45. RISC-V die with power rings defined

We then add stripes to the design with the configuration shown below.

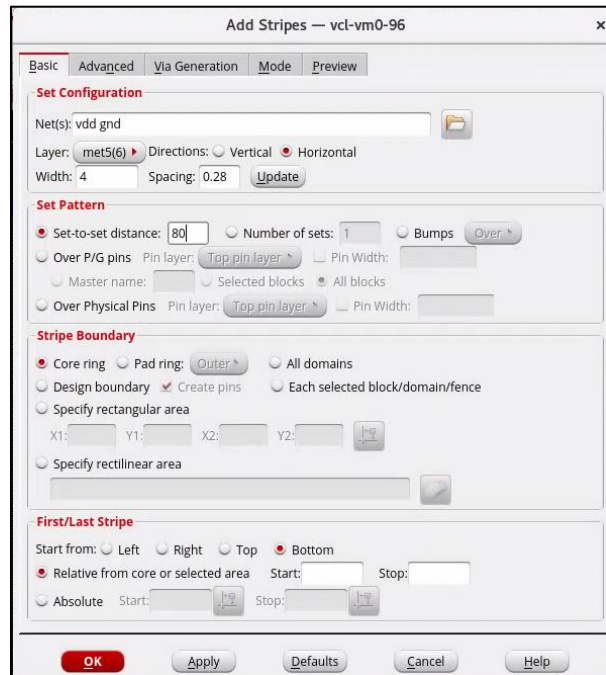


Figure 46. Power stripes configuration

After clicking OK, we can see the stripes added to the design. The schematic has more stripes compared to what we had in Problem 1. This is expected since the cell area is larger for SkyWater, while the distance between the stripes is unchanged from Part 1.

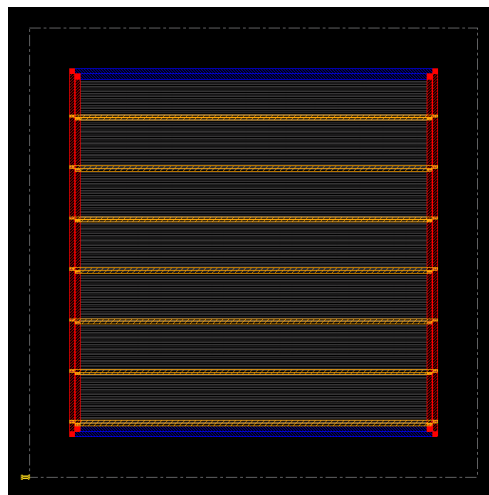


Figure 47. RISC-V die with power rings and stripes defined

Step 5 – Placement

We use the same TCL commands to perform the placement operation:

```
createBasicPathGroups -expanded  
get_path_groups  
place_opt_design
```

After waiting about 30 minutes, we get the following schematic:

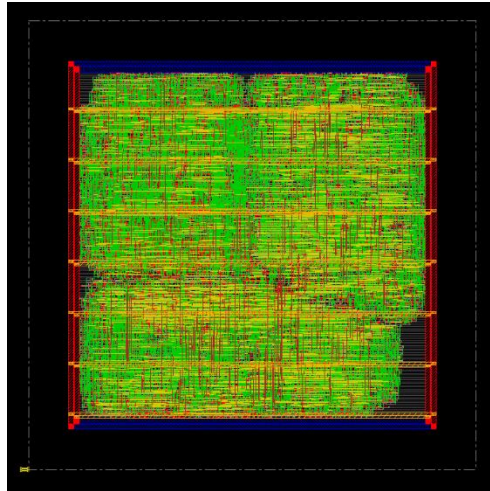


Figure 48. Stick diagram for the RISC-V design implemented using FreePDK45

Step 6 – Pre-CTS Timing Analysis and Optimization

We ran the Pre-CTS Timing Analysis as before:

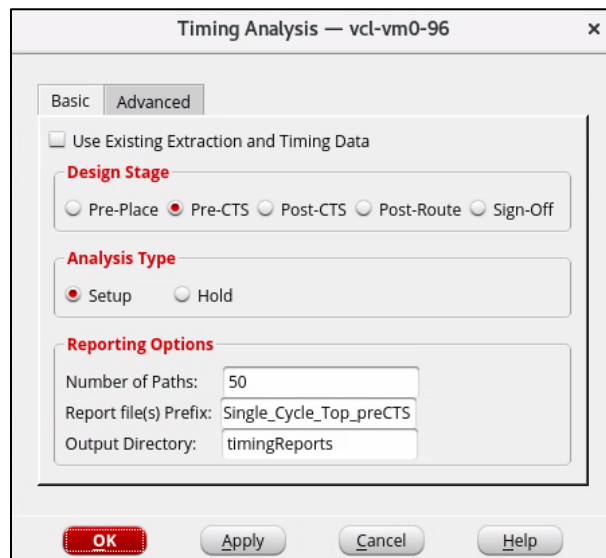


Figure 49. Generating pre-CTS timing analysis report

timeDesign Summary						

Setup views included:						
test_av						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	0.000	0.000	0.959	8.174	N/A	0.000
TNS (ns):	0.000	0.000	0.000	0.000	N/A	0.000
Violating Paths:	0	0	0	0	N/A	0
All Paths:	5302	5237	1103	32	N/A	0

DRVs	Real		Total			
	Nr nets(terms)	Worst Vio	Nr nets(terms)			
max_cap	0 (0)	0.000	0 (0)			
max_tran	0 (0)	0.000	0 (0)			
max_fanout	0 (0)	0	0 (0)			
max_length	0 (0)	0	0 (0)			

Density: 69.417%						
Routing Overflow: 0.17% H and 0.00% V						

Reported timing to dir timingReports						
Total CPU time: 1.78 sec						
Total Real time: 3.0 sec						
Total Memory Usage: 2142.359375 Mbytes						
*** timeDesign #1 [finish] : cpu/real = 0:00:01.8/0:00:03.0 (0.6), totSession cpu/real = 0:18:18.1/1:44:24.9 (0.2), mem = 2142.4M						

Figure 50. Pre-CTS setup timing analysis report

Since there is no negative slack, there is no need to run any optimization here.

Step 7 – Clock Tree Synthesis (CTS)

Before performing the Clock Tree Synthesis (CTS), clock latency, uncertainty, and ideality must be commented in the SDC file (if any) and sourced again in the tool by invoking the following command:

```
update_constraint_mode -name test_cm -sdc_files Single_Cycle_Top_syn.sdc
```

After we source the new SDC file, we can run the following commands to create a clock tree specification and configure CCOpt library cells.

```
create_ccopt_clock_tree_spec -file ccopt.spec
source ccopt.spec
```

Step 8 – Post-CTS Timing Analysis

Having done the Clock Tree Synthesis, we then run another round of optimization, this time specifying that it is post-CTS:

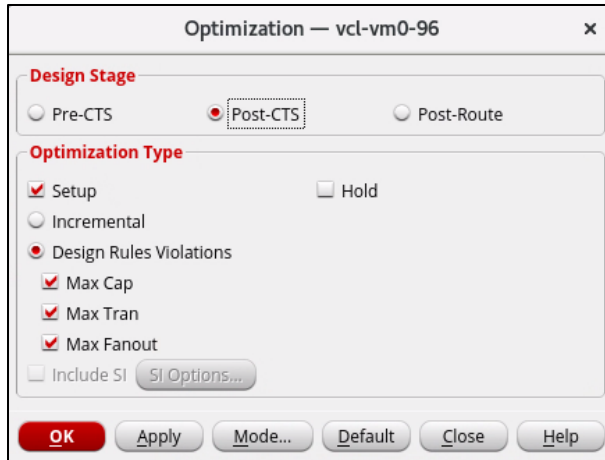


Figure 51. Generating post-CTS timing analysis report

Here are the results:

optDesign Final Summary							

Setup views included:							
test_av							

Setup mode	all	reg2reg	in2reg	reg2out	in2out	default	
WNS (ns):	-0.000	-0.000	0.881	8.179	N/A	-0.000	
TNS (ns):	-0.000	0.000	0.000	0.000	N/A	0.000	
Violating Paths:	1	1	0	0	N/A	0	
All Paths:	5302	5237	1103	32	N/A	0	

DRVs	Real		Total				
	Nr nets(terms)	Worst Vio	Nr nets(terms)				
max_cap	0 (0)	0.000	0 (0)				
max_tran	0 (0)	0.000	0 (0)				
max_fanout	0 (0)	0	0 (0)				
max_length	0 (0)	0	0 (0)				

Density: 70.237%							
Routing Overflow: 0.09% H and 0.00% V							

Figure 52

We can see that there are two violating paths, but the negative slack is so miniscule that it appears as – 0.000.

Step 9 – NanoRoute

We open up NanoRoute:



Figure 53. NanoRoute configuration window

We then press OK.

Step 10 – Post-Routing Timing Analysis (Report Critical Path and Maximum Frequency)

As in Part 1, we first run the following:

```
setAnalysisMode -analysisType onChipVariation -cpr both
```

Then we generate the timing report as shown before, with the Design Stage set to Post-Route.

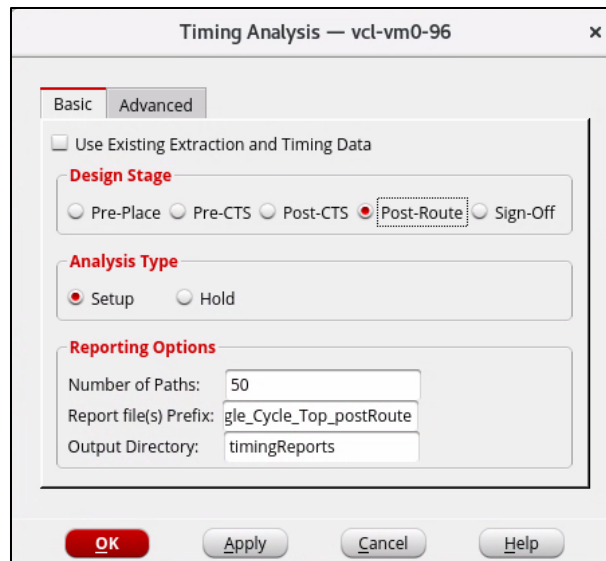


Figure 54. Generating post-route timing analysis report

Here is the result. We can see that there are many violating paths:

timeDesign Summary						
Setup views included:						
test_av						
Setup mode	all	reg2reg	in2reg	reg2out	in2out	default
WNS (ns):	-13.722	-0.613	-13.722	7.556	N/A	0.000
TNS (ns):	-13989.2	-236.861	-13989.2	0.000	N/A	0.000
Violating Paths:	1070	839	1070	0	N/A	0
All Paths:	5302	5237	1103	32	N/A	0
DRVs	Real		Total			
	Nr nets(terms)	Worst Vio	Nr nets(terms)			
max_cap	0 (0)	0.000	0 (0)			
max_tran	26 (102)	-13.478	26 (106)			
max_fanout	0 (0)	0	0 (0)			
max_length	0 (0)	0	0 (0)			
Density: 70.237%						

Figure 55. Post-route setup timing analysis report

Step 11 – Output Data

We ran the same commands as in Part 1. It generated the SDF, GDS, and netlist file.

Step 12 – Design Metrics (Area and Power)

We ran the “report_area” command to get the following area usage report:

innovus 20> report_area			
Hinst Name	Module Name	Inst Count	Total Area

Single_Cycle_Top		18172	208102.086
ALU	ALU	832	4913.462
ALU/DW01_addsub_inst	ALU_DW01_addsub_1	540	3355.718
Control_Unit_Top	Control_Unit_Top	36	207.699
Control_Unit_Top/ALU_Decoder	ALU_Decoder	8	43.792
Control_Unit_Top/Main_Decoder	Main_Decoder	28	163.907
Data_Memory	SRAM_32x64_1rw_0	5882	71990.294
Instruction_Memory	SRAM_32x64_1rw_1	5886	72253.046
Mux_DataMemory_to_Register	Mux_0	34	351.587
Mux_PCNext	Mux_2	26	157.651
Mux_Register_to_ALU	Mux_1	124	910.874
PC	PC_Module	24	265.254
PC_Adder_4	PC_Adder_1	9	155.149
PC_Adder_4/add_35	PC_Adder_1_DW01_add_0_DW01_add_1	9	155.149
PC_Adder_Imm	PC_Adder_0	18	202.694
PC_Adder_Imm/add_35	PC_Adder_0_DW01_add_0	18	202.694
Register_File	Register_File	5265	56495.434
Sign_Extend	Sign_Extend	17	115.110
innovus 21>			

Figure 56. Post-implementation area usage report

The area is 208102.086 units, which is about ten times larger than the resulting device we implemented in FreePDK. Next, we run “report_power” to generate the power usage report:

Total Power						

Total Internal Power:		30.64320922		85.4455%		
Total Switching Power:		5.21957526		14.5543%		
Total Leakage Power:		0.00007903		0.0002%		
Total Power:		35.86286351				

Group		Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)

Sequential		27.78	1.448	4.658e-05	29.23	81.5
Macro		0	0	0	0	0
IO		0	0	0	0	0
Combinational		2.863	3.772	3.245e-05	6.635	18.5
Clock (Combinational)		0	0	0	0	0
Clock (Sequential)		0	0	0	0	0

Total		30.64	5.22	7.903e-05	35.86	100

Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)

Default	1.8	30.64	5.22	7.903e-05	35.86	100

* Power Distribution Summary:						
* Highest Average Power:		Register_File/FE_OF638_Result_19 (sky130_fd_sc_hd_inv_4):			0.01834	
* Highest Leakage Power:		ALU/U233 (sky130_fd_sc_hd_nor2_8):			3.044e-08	
* Total Cap:		2.06074e-10 F				
* Total instances in design:		18172				
* Total instances in design with no power:		0				
* Total instances in design with no activity:		0				

* Total Fillers and Decap:		0				

Figure 57. Post-implementation power consumption report

The total power usage is 35.86mW, which is about four times greater than with FreePDK.

Problem 2.5 – Post-routing Parasitic (SDF) back-annotation (Gate-level Simulation)

Run a post-routing simulation on your RISC-V design using a multiplication program.

To launch VCS with the generated netlist, we used this command:

```
vcs -kdb -full64 -debug_access -r +v2k +neg_tchk
./rtl/sim_1/Single_Cycle_Top_Tb.v Single_Cycle_Top_syn.v
./skywater/stdcells.v ./skywater/primitives.v
```

Unfortunately, because we couldn't use the OpenRAM memory, the synthesis generated a netlist for the instruction memory with names that did not match the testbench. It would be impractical to change the testbench to match every single changed name, so we were not able to run the simulation.

Part 3: Run-in-batch Synthesis Manager

Problem

Write a script (c-shell, bash, python, etc.) that synthesizes a design several times with different parameter values as indicated by the user and generates a CVS (comma-separated) table listing the area, power, and maximum frequency for each parameter value. For this step, you may assume that the design is the parametric double dabble binary to BCD converter.

We first had to modify the Verilog code by registering the inputs and outputs. We also had to add a clock to the design to calculate the maximum clock frequency later.

To do this, we created two new inputs, `i_clk` and `i_rstn`, to function as a clock and reset input. We created an internal register `bin_reg`, which stores the input binary value to convert to BCD. We also created an internal register, `bcd_reg`, which stores the resulting BCD value and is tied to the BCD output. By adding clocked elements to the inputs and outputs, we ensure that the synthesis tools treat the device as a sequential circuit.

After making these changes, we tested the Verilog code to ensure it is still working as intended. We, therefore, created a testbench called `bin2bcd_tb.v`. This testbench converts some binary values into BCD. We then look at the output and see that it is correct:

```
If you would like to temporarily disable this message, set
the VCS_LIC_EXPIRE_WARNING environment variable to the number of days
before expiration that you want this message to start (the minimum is 0).
-----
input_binary(DECIMAL) = 8, output_BCD = 1000
-----
input_binary(DECIMAL) = 12, output_BCD = 10010
-----
input_binary(DECIMAL) = 7, output_BCD = 111
-----
input_binary(DECIMAL) = 2, output_BCD = 10
-----
Ali Abbasi, Derek Furtado
ECE748@McMaster - Fall2023
-----
$finish called from file "sim_1/bin2bcd_tb.v", line 99.
$finish at simulation time 180000
V C S   S i m u l a t i o n   R e p o r t
Time: 180000 ps
CPU Time: 0.300 seconds;      Data structure size: 0.0Mb
Thu Dec 21 17:00:32 2023
```

Figure 58. Simulating the Binary-to-BCD converter module with inputs and outputs registered

Now that we have verified this, we can synthesize the code and find the Fmax value. The script we used to synthesize the Verilog code in Design Compiler is called `part_3.tcl`. We use `freepdk45` for this experiment, so we set our `target_library` and `link_library` to include the same `gsc145nm.db` that we used in Part 1. Because we are mainly concerned with the timing constraints, we set the maximum capacitance to a high value (100000) so that the maximum capacitance is satisfied even for large parameter (`W`) values. We set our clock in the script to be `i_clk`, which is the signal name we added in the Verilog code.

In order to calculate the Fmax, we need to find the smallest clock period for which the synthesized code satisfies the timing constraints. This means that for a given parameter value, we follow these steps:

- Modify the TCL script to select a clock period of 0.1

- Synthesize the design and check if it meets all constraints
- If not, increment the clock period by 0.1 and try again
- If it does, take the reciprocal of this clock period for Fmax

Note that this algorithm is a linear search: we increment our clock period step-by-step until the design synthesizes without any violations. Once we succeed, we take this clock period and calculate the frequency. This is a straightforward algorithm to implement, but it can be slow for larger designs. Ideally, we would craft a more involved algorithm using a binary search to find the minimum possible clock period. However, this would increase the script complexity by a lot.

Our resulting script is contained in two files: `part_3.csh` and `part_3_bash.sh`. In order to test this script, run the CSH script as shown below. The full BASH script can be seen in Figure 59.

```
./part_3.csh [parameters]
```

The CSH script is used to source the Design Compiler. It then invokes the bash script to run the main search algorithm and data processing. Once the script finds the minimum successful clock period, it scrapes the area, timing, and power information from the `dc_shell` output and writes it to the CSV file `results.csv`. It then continues this process with all the other parameters entered by the user. Here is an example csv output from the script “`./part_3.csh 7 9 13 16 22`” (which was completed in about an hour):

Param	Fmax	Area	Power
7	1429	330.86	329.03
9	1000	638.25	342.67
13	714	1133.36	419.88
16	556	1710.60	480.69
22	417	3493.47	728.53

Figure 59. Example output of the developed in-bash synthesis manager for Binary-to-BCD converter module

```

#!/bin/bash
# check if argument supplied
if [ $# -eq 0 ]; then
    echo "Please enter your parameters as command line arguments"
    exit 1
fi

# create output file
echo "Param, Fmax, Area, Power" > results.csv

# loop through supplied parameters
for param in "$@"; do
    echo "parameter: $param"
    # modify testbench file to change parameter
    sed -i "s/W = [0-9]\+/W = $param/" bin2bcd.v
    # run file and scrape statistics
    clk_period=0.1
    while true; do
        echo "testing clock period: $clk_period"
        sed -i "s/^create clock -period.*$/\
create_clock -period $clk_period i_clk/" part 3.tcl
        dc_shell -f part 3.tcl > log.tmp
        max_c=$(cat log.tmp | grep 'max_capacitance' | tail -n 1 \
| awk '{print $3}')
        max_ds=$(cat log.tmp | grep 'max_delay/setup' | tail -n 1 \
| awk '{print $3}')
        seq_clk_w=$(cat log.tmp | grep 'sequential_clock_pulse_width' \
| tail -n 1 | awk '{print $3}')
        c_range=$(cat log.tmp | grep 'critical_range' | tail -n 1 \
| awk '{print $3}')
        if [[ "$max_c" == "(MET)" && \
"$max_ds" == "(MET)" && \
"$seq_clk_w" == "(MET)" && \
"$c_range" == "(MET)" ]]; then
            # passing condition; calculate Fmax
            echo "conditions passed, writing to file"
            #f_max=$(printf "%.0f" "echo "scale=2; (1 / $clk_period)*1000" | bc)
            f_max=$(printf "%.0f" \
"$($echo "scale=10; (1 / $clk_period) * 1000" | bc)")
            # also get Area
            area=$(cat log.tmp | grep 'Total cell area' \
| awk '{printf "%.2f\n", $NF}')
            # get Power
            power=$(cat log.tmp \
| grep 'Total Dynamic Power' \
| awk '{printf "%.2f\n", $5}')
            # print line to file
            echo "$param, $f_max, $area, $power" \
>> results.csv
            break
        fi
        clk_period=$(echo "$clk_period + 0.1" | bc | sed 's/^\.0./')
    done
done

```

Figure 60. In-bash synthesis manager script developed for the Binary-to-BCD converter